

A Systematic Literature Review on Vulnerability Detection Approaches for IoT Mobile Applications

Zongo Meyo, Rodrigo Morales, Ildiko Pete, Yann-Gaël Guéhéneuc

Abstract—Internet of Things (IoT) systems are pervasive and increasingly managed through mobile applications. However, poorly designed mobile applications can expose sensitive information to external adversaries. Mitigating such vulnerabilities requires both developers and researchers to apply well-established practices and design secure systems based on clearly defined approaches for vulnerability detection. Although databases such as Open Worldwide Application Security Project (OWASP) and Common Vulnerabilities and Exposures (CVE) catalog known IoT vulnerabilities, no standardised methodology exists for detecting security weaknesses in IoT Mobile applications (IoTMAs) during IoT mobile application development. Building on prior research, our research objectives are to: (1) identify, classify, and prioritize critical security vulnerabilities in IoTMAs, (2) survey existing Vulnerability Detection Approaches (VDAs) for IoT mobile applications, (3) critically evaluate the effectiveness of existing VDAs by analyzing their evaluation methodologies and dataset validation, and (4) formulate evidence-based recommendations based on the limitations of existing methods for detecting security vulnerabilities in IoTMAs. We performed a systematic literature review (SLR) from selected primary studies (PSs). From 856 papers retrieved from six academic databases—Scopus, Springer, and Engineering Village, which hosts Compendex (covering IEEE Xplore and the ACM Digital Library), and Inspec (IET)—we reviewed 39 research papers. Our findings include: (1) identification of 52 security vulnerabilities, eight critical (i.e., reported in at least four studies); (2) discovery of seven distinct VDAs; (3) comprehensive VDAs effectiveness evaluation using empirical metrics, accuracy assessments, reproducibility analysis, comparative studies, and validation across diverse IoTMAs marketplaces; (4) recommendations to guide developers and practitioners in selecting appropriate VDAs, thereby supporting the development of secure IoTMAs and enhanced penetration testing. Our study raises awareness of state-of-the-art VDAs, identifies research gaps in existing approaches, and provides recommendations to enhance existing techniques and guide new development, supporting software engineers in making informed technique selection decisions.

Index Terms—IoT Mobile applications (IoTMAs), security vulnerabilities, vulnerability detection approaches (VDA).

Zongo Meyo is with the Department of Computer Science and Software Engineering, Concordia University, Montréal, Canada
Email: minette.zongomeyo@concordia.ca.

Rodrigo Morales is with the Department of Computer Science and Software Engineering, Montréal, Canada
Email: rodrigo.moralesalvarado@concordia.ca.

Ildiko Pete is an independent researcher, Zurich, Switzerland
Email: pildip23@hotmail.com.

Yann-Gaël Guéhéneuc is with the Department of Computer Science and Software Engineering, Montréal, Canada
Email: yann-gael.gueheneuc@concordia.ca.

I. INTRODUCTION

IoT systems have gained significant traction in the past two decades [1]. They are increasingly embedded in daily life and across society. The growing penetration of IoT devices is expected to enhance infrastructure, public utilities, and services, while also driving the development of smart cities. These advancements enable improved decision-making and foster innovation in addressing complex global challenges [2].

In the case of smart home systems, voice assistants such as Amazon Alexa, Apple Siri, and Google Home promise solutions to challenges related to health, aging, and energy consumption [3]. They also offer smart TVs, smart refrigerators, humidity sensors, doorbells, thermostats, etc. to improve comfort and automate daily tasks.

However, alongside their great potential, IoT devices also introduce significant risks. These devices are now widely integrated into homes and organizations, where their functionality is managed through IoTMAs. These applications serve as the primary control interface, meaning that anyone with access, whether legitimate owners or malicious third parties, can operate the connected devices. Therefore, ensuring robust security is essential for users to safely benefit from IoT systems.

Insecure applications on IoT devices pose a risk of data breaches. In many smart home scenarios, a single device can become an entry point for exploitation. In the case of premeditated security vulnerabilities (e.g., weak, easily guessable, or hard-coded passwords), the collected data can pose a threat to society or to specific groups of people [4].

Preventing such vulnerabilities requires developers and researchers to understand secure application design principles. These principles can be established by systematically assessing applications security through vulnerabilities detection methods. This, in turn, requires developing effective approaches for identifying vulnerabilities in IoT applications, called vulnerability detection approaches (VDA).

Software engineers have access to a wide range of VDAs for detecting vulnerabilities in IoT applications. However, a gap remains between industry and academia: practitioners are often unaware of the full spectrum of available VDAs and their characteristics, while many academic approaches are not readily applicable to industrial systems. To the best of our knowledge, no previous study has systematically reviewed existing VDAs for IoTMAs, assessed their strengths and limitations, and offered practical recommendations to support secure IoT application development in both academic and industrial contexts..

Consequently, we conduct a systematic literature review of VDAs applied to IoTMA to: (1) identify the most prevalent security vulnerabilities in IoTMA, and (2) analyze their VDAs based on the methodology, evaluation metrics, and validation datasets. We review 39 primary studies (PSs) out of 856 initial candidate papers found in the following academic databases: Engineering Village hosting Compendex and Inspect-IET, Scopus, and Springer.

We aim to investigate four research questions (RQs), and we summarise the contributions of this work as follows:

- We present a systematic literature review of the most prevalent security vulnerabilities in IoTMAs, classifying them according to the OWASP 2021 Top 10, the current standard, pending the forthcoming 2025 release. To address the mobile-specific security landscape of IoTMAs, we refine and extend this classification to OWASP MASVS v2.1.0, the industry standard for mobile application security, which was released in January 2024.
- We present a state-of-the-art description of VDAs used to detect IoT security vulnerabilities in mobile applications.
- An analytical summary of the effectiveness of VDAs measured across evaluation metrics and validation datasets.
- Evidence-based recommendations derived from our analysis to guide IoT researchers and practitioners in developing and assessing IoTMA security across the entire development lifecycle.

The remainder of the paper is as follows. Section II presents the background on IoTMAs in IoT systems, and introduces vulnerability issues present in IoTMAs. In Section III, we review related research. Section IV describes our study design. Section V presents the results of our study. Section VI discusses threats to validity of our results, while Section VII concludes the paper and outlines future work.

II. PRELIMINARIES ON IOT MOBILE APPLICATIONS

A. Overview of IoT Mobile Applications

IoT systems are applied across multiple domains and increasingly integrated into everyday life. From smart cities to Intelligent Transportation Systems, they enhance the quality of life for many individuals. IoTMAs serve as the main interface to manage, monitor and control IoT devices in IoT systems, such as smart homes, industrial IoT, healthcare, and other smart environments. They facilitate user interaction with IoT devices—such as thermostats, wearables, cameras, curtains—by providing remote access, configuration, or real-time data visualisation.

The proliferation of IoTMAs has revolutionised how people interact with networked devices. One of the key advantages of such apps is that they promote pervasive connectivity, enabling users to access, manage, and control IoT devices remotely, free from temporal or geographical limitations. The constant availability model significantly increases user convenience and promotes more timely responses to system states or environmental stimuli [5].

These applications enable the rapid detection of performance anomalies and the prompt execution of corrective actions. The inter-operation of automation functions via mobile

controls minimizes manual intervention, streamlines workflow, and improves IoT deployment efficiency overall. This is particularly evident in smart home and industrial environments where operational responsiveness is critical [6].

Additionally, IoTMAs facilitate run-time adaptations in the behavior of devices and networks according to the current scale and state of networked assets. Through the support of decentralised control and cooperative systems, they promote system resilience and effective use of resources. This aspect is consistent with long-term sustainability objectives, where IoTMAs are utilized in smart homes and cities to realize energy conservation and environmental footprint reduction [5], [6].

IoT Mobile Applications connect users to IoT ecosystems via protocols such as Message Queuing Telemetry Transport (MQTT), HyperText Transfer Protocol (HTTP), secure APIs, among others. They are deployed across diverse platforms, making their security a crucial concern [7]. In fact, the proliferation of IoTMAs poses serious security and privacy challenges, driven by the complexity of IoT environments and the resource constraints of connected devices. These factors expose systems to a wide range of cyber threats, including malware, data breaches, and unauthorized access. Several studies highlight the need for effective security mechanisms tailored to IoTMAs, including anomaly detection, intrusion prevention systems, and secure communication protocols [5].

B. Security vulnerabilities in IoT Mobile Applications

The security of IoTMAs is governed by established security standards and guidelines that define the baseline requirements and best practices. In fact, OWASP advised companies to adopt their standard awareness document to ensure that their web applications minimize security risks. They continue by stating that the OWASP Top 10 is perhaps the most effective first step towards changing the software development culture within an organization into one that produces more secure code [8]. OWASP foundation has also released the OWASP Mobile Application Security (MAS) flagship project, which provides a security and privacy standard for mobile apps, called OWASP Mobile Application Security Verification Standard (MASVS). OWASP MASVS is a collection of mobile app-specific weaknesses and a comprehensive testing guide that covers the processes, techniques, and tools used during a mobile app security test, as well as an exhaustive set of test cases that enables testers to deliver consistent and complete results [9].

According to Reddy et al. [5], IoTMA security vulnerabilities are primarily a result of the heterogeneous and complicated nature of IoT environments. The fragmentation of device types, communication standards, software platforms, and protocols creates a complex environment where security cannot be implemented uniformly. This fragmentation increases the attack surface by introducing loopholes that malicious parties can capitalize on, especially given that the majority of IoT deployments lack security from the design to the deployment stage.

Reddy et al. [5] note that the ongoing evolution of IoT standards and architectures also complicates security manage-

ment since new devices and protocols introduce new, untested vulnerabilities. The coexistence of legacy systems alongside newer devices aggravates interoperability deficiencies that increase security risks by their very nature.

They also list wireless communication techniques — central to IoT device connectivity— as a source of threats in IoTMs. In fact, some wireless communication techniques expose transmitted data to interception, alteration, and replay threats. Since numerous IoT devices use open or weakly protected wireless channels, attackers have the chance to violate confidentiality and integrity via eavesdropping or man-in-the-middle (MITM) attacks.

Schiller et al. [10] have also listed security vulnerabilities in IoTMs:

- Weak, guessable, or hard-coded passwords are a major concern, as many users do not change default credentials, and some IoT devices have non modifiable hard-coded credentials. This allows unauthorized access to devices, and home networks via attacks such as data theft and malware infections.
- Insecure network services and ecosystem interfaces are considered equally significant vulnerabilities. For instance, a lack of secure update mechanisms and the use of insecure or outdated components worsen the risks.
- Insufficient privacy protection, insecure data transfer and storage, and lack of good device management procedures contribute to the overall vulnerability of smart home systems.
- Poor physical security and insecure default settings equally make devices prone to compromise.
- Generic IoT technologies are often characterized by unencrypted data transmissions and weak passwords.

These vulnerabilities are of a great concern because they can lead to various attacks, including malware/botnet infections, denial-of-service attacks, scanning attacks, and compromised applications.

III. RELATED WORK

A. Background

In security terminology, the Common Weakness Enumeration (CWE) defines a weakness as a flaw in software design, implementation, or architecture that can potentially lead to security issues, while the Common Vulnerabilities and Exposures (CVE) defines a vulnerability as is a specific, exploitable instance of a weakness in a particular system.

In this study, We define a security vulnerability as both underlying security weaknesses-such as design or implementation flaws-and their exploitable manifestations in specific IoTMs. Following the OWASP terminology, these represent failures to meet security requirements across the categories defined by OWASP. Our focus in this study is to identify classes of security weaknesses prevalent in IoTMs.

B. Related Studies

To the best of our knowledge, a comprehensive assessment of VDA on IoTMs has not yet been produced. Following is

a brief discussion of work that is closely related to the present study.

Svacina et al. [7] conducted a systematic literature review of log analysis techniques for detecting security issues and system intrusions, analyzing 34 relevant papers from 1,374 publications (2010–2020). The review covered various software systems, including enterprise systems (e.g., Active Directory and networks), control software (e.g., a log mining approach for process monitoring in SCADA), and IoT components, where security vulnerabilities are critical. The authors found that although many studies exist on log analysis, few address both the analysis process and security aspects, and previous similar reviews are outdated, with the most recent covering work only up to 2016. The study identified two primary detection approaches: signature-based and anomaly-based detection, and recommended a hybrid approach combining both to enhance detection accuracy. Svacina et al. highlighted IoT security vulnerabilities, but focused on infrastructure-level log analysis rather than the mobile applications that serve as the main user interface.

Reddy et al. [5] systematically analyzed the application of Swarm Intelligence (SI) algorithms in designing Intrusion Detection Systems (IDS) for IoT environments. The study compiles and evaluates SI-based optimised techniques from 2010 to 2023, by reviewing over 100 papers and classified approaches based on system deployments, detection methodology, and optimization strategy. The authors tackle the challenge of building effective and adaptive IDS for IoT networks, which are dynamic and prone to security vulnerabilities. The authors identified SI techniques and their roles in IDS development, including SI-based algorithms, hybrid approaches, optimisation goals, and evaluation metrics. While their review is highly relevant in terms of IDS optimisation for IoT, it overlooks mobile application vulnerabilities and threat models.

Alshamsi et al. [6] investigated the application of machine learning (ML) techniques for malware detection in smart homes environments through a systematic literature review of 56 papers published between 2019 and 2023. The study examined security threats in smart homes IoT networks and assessed various artificial intelligence (AI) and ML-based detection strategies, including both static and dynamic analysis techniques. Their results contributed to classifying ML-based malware detection techniques into three major types, including anomaly-based detection, ensemble learning, and deep learning (DL). They identified two analysis approaches: static and dynamic, for malware detection using AI and ML algorithms. While the study provides a comprehensive synthesis of ML techniques for malware detection in smart home environments, it focuses on primary device-level threats and network-based intrusions, and one specific security vulnerability. But the authors overlooked vulnerabilities introduced by IoTMs.

We have reviewed the three above studies and confirmed that none of these studies directly address the growing security concern associated with IoTMs that serve as gateways for interacting with devices in an IoT ecosystem. IoTMs introduce unique vulnerabilities that require distinct detection approaches. To fill this gap, our study conducts an SLR review on security VDAs. By focusing on the application layer,

our study will expand the current landscape of IoT security research and provide tailored insights to guide IoT researchers and practitioners in developing and assessing secure IoTMA.s.

IV. STUDY DESIGN

The study design is built upon our previous work [11] and follows a combination of guidelines by Kitchenham et al. [12], [13], and by PRISMA [14] for SLRs. The study consists of the following steps: (1) formulating research questions (RQs), (2) devising a search strategy, (3) selection procedure and quality assessment, and (4) data extraction.

A. Research Questions

The objective of this review is to identify and characterize VDAs, with respect to the most prevalent security vulnerabilities in IoTMA.s.

Our research questions are based on our objective of understanding the current state of VDAs along with existing challenges, which we have mentioned in the introduction. We use the answers to our RQs to formulate recommendations for researchers and practitioners on developing and adopting VDAs to develop secure mobile applications and perform comprehensive penetration tests.

- (RQ1) **What are the most prevalent security vulnerabilities in IoTMA.s based on the IoT threat landscape?**

Rationale: this question enables us to systematically identify the predominant security vulnerabilities in IoTMA, and rank them with respect to their frequency of occurrence in our PSs.

- (RQ2) **Which methods are used to detect vulnerabilities in IoTMA.s?**

Rationale: this question will provide researchers and practitioners with existing techniques to detect security vulnerabilities in IoTMA.s.

- (RQ3) **Which validation metrics and datasets are commonly applied in VDAs for IoTMA, and how effective are they in assessing VDAs?**

Rationale: this question will yield insights essential for comparing approaches and determining their reliability for specific vulnerability types.

- (RQ4) **What are the recommendations to guide researchers and practitioners in improving security in IoTMA.s?**

Rationale: this question will provide best practices derived from identified gaps, highlighting ways to bridge theory and practice while providing guidance to both researchers and industry professionals.

B. Search Strategy

1) *Database selection:* We conducted the literature search through the digital meta-library Engineering Village¹ (EV). EV offers a unified access to 12 engineering literature and patent databases, including the main engineering sources

(ACM, Compendex, EI IEEE, Inspect-IET, and Springer). To ensure full coverage of relevant literature, we conducted an identical query in Scopus, targeting research papers published between 2022 and 2024. This supplementary search yielded no additional papers, and it led us to conclude that EV database provided sufficient coverage for our literature search objectives at the query formulation stage.

2) *Query formulation:* We searched in the title, abstract, keywords fields using the following query:

```
((iot) WN ALL) AND ((mobile app*) WN ALL) AND (((vulnerabilit*) OR exploit OR exploits) WN ALL) AND (english WN LA) NOT (arxiv WN ST) NOT (cloud WN ALL) AND ((ca OR ja ) WN DT))
```

The motivation to use these keywords stems from the focus of the paper's objectives, which are stated in our introduction.

This stage returns an initial number of candidate papers, and we continue with the literature selection procedure that is described below.

3) *Literature selection procedure:* After the initial stage, we proceeded with the literature selection procedure. The first thing we did was to remove duplicate studies, that is, studies with the same titles, by the same authors, and published in the same venue with the same or similar contributions.

After the removal of duplicate studies, we conducted a screening process, which consisted of applying the inclusion/exclusion criteria to the initial candidate papers after deduplication. Then, we performed a snowballing process on the remaining set.

Our inclusion criteria are defined as follows:

- The paper has been published in conference proceedings or journal issues.
- The paper was written in English.
- The paper presents original work
- The paper discusses vulnerability issues related to IoT Mobile Applications.
- The paper proposes and/or implements a VDA.

Likewise, we defined our exclusion criteria as follows:

- The papers are systematic reviews and mapping studies
- The papers are identical or provide similar contributions (duplicates).
- The papers are theoretical studies
- The papers are position papers, and a study has not yet been carried out
- The papers are not publicly available
- The papers did not undergo peer review.
- Focus on firmware.

The snowballing process can be divided into two distinct processes: forward snowballing and backward snowballing.

Forward snowballing, refers to the citation tracking, which means all sources that have cited the original paper that is being examined. We conducted this citation tracking activity using EV, Google Scholar, and Scopus, until no new studies were found.

Backward snowballing, refers to the tracking of references, that is, using the reference list of a candidate paper to identify potentially relevant articles. It is a straightforward process that

¹<https://www.engineeringvillage.com/>

we conducted by downloading reference lists of all candidate papers, using EV.

Following the snowballing process, we applied the inclusion/exclusion criteria to the newly identified candidate papers, retaining only those that met our study requirements.

4) *Quality assessment*: We assessed our candidate papers on their quality using the following binary questions:

- 1) Is the paper technically sound?
- 2) Is the research method clear?
- 3) Does the proposed method address the objectives set by the authors?
- 4) Is the technical depth of the paper sufficient?
- 5) Is the proposed technique/tool evaluated properly?

Each author independently evaluated papers using binary coding: "1" for inclusion, "0" for exclusion, with "1?" or "0?" indicating uncertainty requiring discussion. Collaborative review was reserved for papers with discordant assessments, i.e., at least one author's evaluation differed from the consensus. Papers receiving unanimous agreement were accepted or rejected without further discussion. In the final stage, we excluded papers that received a "0" rating from authors. All remaining papers, where at least one author assigned a "1," underwent team discussion to determine final inclusion.

5) *Eligibility criteria*: At this stage, we observed disparities in the candidate papers and, upon closer examination, found that some were unrelated to IoTMA, instead focusing on other areas of the IoT ecosystem, such as protocols and networks. So we decided to add a final eligibility criterion stated as follows:

- Does the paper focus on IoT Mobile Applications?

At the end of this stage we obtained our primary studies.

C. Data extraction

Following the literature selection process, we identified our primary studies from which we extracted relevant data for our research.

The data extraction process was conducted, and reviewed by three authors to extract information about the state-of-the-art VDA techniques and their characteristics based on the following features:

- The objective (what the authors wanted to achieve) and the subject of their work (IoT devices, IoT device mobile apps, or Android mobile apps in general).
- The vulnerabilities or security issues addressed by the proposed solutions and whether they can be connected to the security risks of the OWASP 2021 Top 10².
- The contributions of the study: what were the authors contributing concerning the objectives of their study? Were they effective in detecting security vulnerabilities in IoTMA?
- The methodology used to meet their objective(s). What techniques did the authors employ to detect security vulnerabilities in IoTMA? Did they implement their approach as a tool?
- The results of the study. Were the results aligned with the authors' objectives? We were interested in the outcome

of their methodology concerning vulnerability detection in IoTMA.

- The metrics used to evaluate the proposed security VDA and the dataset on which these metrics were validated.
- Limitations, and/or threats to validity. Did the authors encounter any issues while designing, developing, and testing their approach? What were these limitations about?

We also gathered metadata on each study, including publication venue, audience, number of analyzed applications, origin, tool availability, and the target devices.

V. RESULTS

First, we report our search strategy outcomes; then, the findings derived from our systematic data analysis of the data extracted; and finally, we provide answers to our RQs.

A. Search strategy results

We adopted the PRISMA 2020 statement to systematically present the results derived from our literature search strategy. Figure 1 illustrates the process and the results corresponding to each stage of the search methodology.

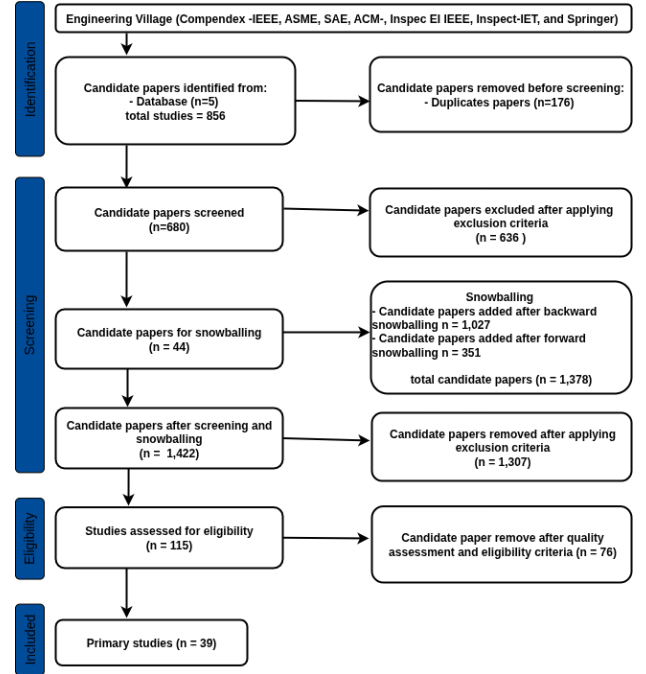


Fig. 1. Primary Studies selection following PRISMA

Candidate papers after the query formulation. Our query returned an initial set of **856** candidate papers.

Candidate papers after de-duplication. After removing potential duplicates, we identified **176** duplicate papers, leaving **680** candidate studies.

Candidate papers after the screening process. Applying the inclusion and exclusion criteria to the **680** candidate papers reduced the set to **44**. We then conducted a snowballing process on these papers, yielding **1,027** additional studies through backward snowballing and **351** through forward snowballing, for a total of **1,422** new candidates. After reapplying the

²<https://owasp.org/www-application.cation>

inclusion and exclusion criteria, we obtained **115** papers after the screening process.

Candidate papers after the quality assessment and eligibility criteria application. After we applied our quality assessment and eligibility criteria, we removed **76** papers.

Selected primary studies (PSs). In total, we selected **39** candidate papers for this systematic literature review.

B. Systematic analysis of extracted data

Keys such as publication year, venue type, target devices, OS platforms, availability of methodology results, etc. are first depicted using graphs. Then the remaining features are analysed in depth.

Evolution of studies by year. Research on IoTMA security analysis began in 2012 and has continued till date, reaching its peak in 2019, with the fewest publications recorded in 2017, as shown in Figure 2

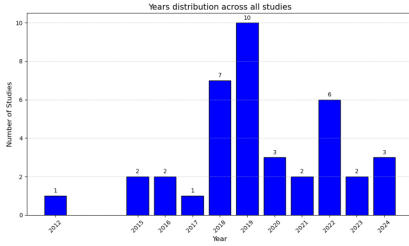


Fig. 2. Years distribution across all studies

Venue type. The major part of PSs falls into two categories: conferences and journals; five at workshops, and three at symposiums, as shown in Figure 3.

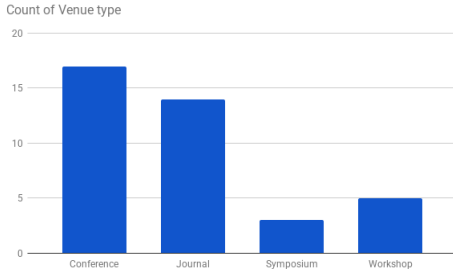


Fig. 3. Academia vs. industry-based work

Target devices of PSs. We identified ten categories of devices, including for example, smart home devices such as smart plugs, smart assistants, smart-cameras, smart toys, and health monitoring devices as illustrated in Figure 4.

Platform or O.S. Figure 5 shows five papers focus on generic platforms, two on generic smart things platforms, one on iOS, two on Alexa platform, while most papers focus on the Android platform.

Number of analyzed apps. Each PSs validated its approach using a varying number and types of apps. As shown in Figure 6, we see some PS validated their approach with a large number of apps-71,000-, others with only a few, and some did not report any app validation.

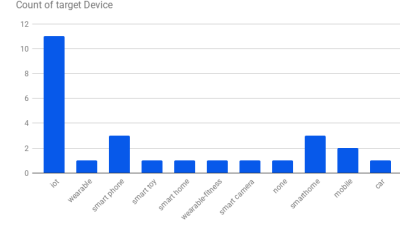


Fig. 4. Target device count

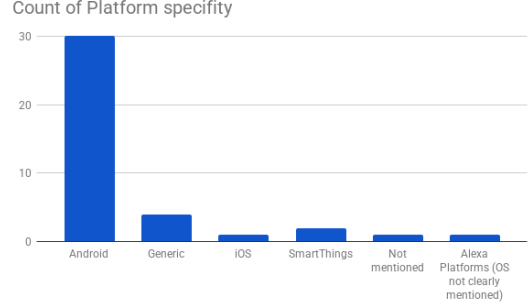


Fig. 5. Platform count

Tool availability. We found that VDA tools were provided in seven cases, while the remaining studies did not make their tools publicly available, as illustrated in Figure 7.

Target stakeholders. Our PSs' authors targeted the following categories of stakeholders: health insurance companies, employers, data intelligence companies, investigators, users, software developers, researchers, and manufacturers; with the top three target stakeholders being researchers, developers, and users, as illustrated in Figure 8.

Applied methodology. We classified each study based on its applied research method—evaluation, validation, or none—to determine whether the reported results were actually implemented or tested, as shown in Figure 9.

C. Answers to RQs

In our previous subsection, we presented findings from straightforward features of the analysis. The rest of the features we have analyzed are more qualitative and will help to answer our RQs.

1) Security vulnerabilities in IoTMA: We have purposely decided to identify and study only the most prevalent security vulnerabilities in IoTMA from our PSs. Although Figure 10 shows all the vulnerabilities we identified, we are only interested in the most prevalent ones. We have manually analyzed each paper thoroughly by reading and extracting targeted information. The results of this analysis are shown in Figure 10, for all detected vulnerabilities, while Table I presents only the most prevalent ones.

Note that each PS addresses more than one security vulnerability. Thus, the total number of studies marked at the edge of each security vulnerability exceeds 39. If we consider the study e.g.,[25], we notice that the authors of the study have reported

TABLE I
CRITICAL SECURITY VULNERABILITIES IN IOTMAS

N°	Security vulnerability name	Studies	Number of papers
1	Unauthorized Access and Unauthorized Data Access	[15], [16], [4], [17], [18], [19], [20], [21], [22], [23], [24], [25]	12
2	Insecure or improper SSL/TLS implementation	[26], [27], [28], [29], [30], [31]	6
3	SQL injection-related vulnerabilities	[32], [26], [33], [25], [34]	5
4	Weak or unencrypted communications	[17], [35], [36], [27]	4
5	Use of broken or risky cryptographic algorithms and Clear text storage	[4], [26], [28], [32]	4
6	Information leakage due to insecure BLE design	[23], [37], [24], [38]	4
7	Insecure device and communication configuration	[39], [22], [25], [35]	4
8	PII leakage due to access control failures	[23], [24], [38], [30]	4

Distribution of the number of analysed apps

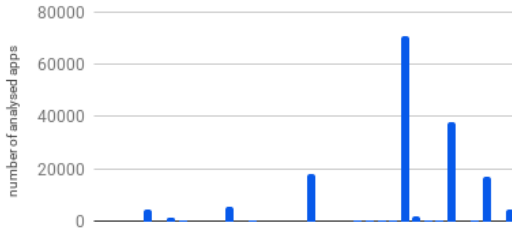


Fig. 6. Distribution of the number of apps analyzed across PSs

Count of tool availability

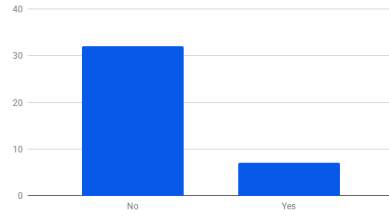


Fig. 7. Tool availability

both unauthorized access and unauthorized data access, and SQL injection-related vulnerabilities as security vulnerabilities in IoTMAS. Similarly, there are many other studies among the PSs that address more than one security vulnerability.

From our analysis, we draw the following conclusions:

Unauthorized Access and Unauthorized Data Access. is the most prevalent vulnerability within IoTMAS, with 12 different studies reporting it as one of the main security vulnerabilities. It encompasses cases where third-parties obtain access to an IoT system or device. For instance, in [21], the authors have detected vulnerabilities related to unauthorized access in the IoT device by only installing the SmartApp, which requires limited access to the device.

Insecure or improper TLS/SSL implementation. comes next with six different studies reporting it as one of the main security vulnerabilities. It encompasses improper certificate validation, security misconfigurations, etc. Such vulnerabilities may result in data leakage.

SQL injection-related vulnerabilities. is found to be the third most detected security vulnerability among our PSs. This

Count of target stake holder

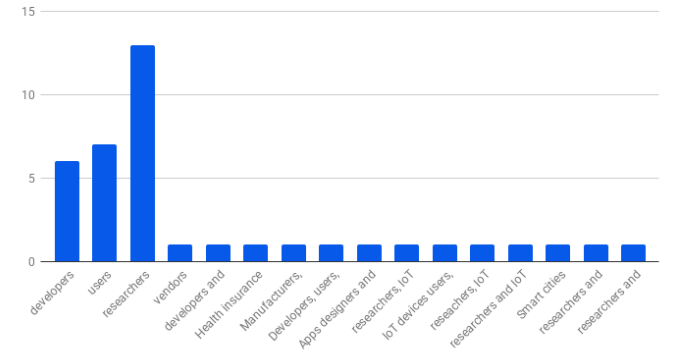


Fig. 8. Target stakeholder count

Count of Applied research method

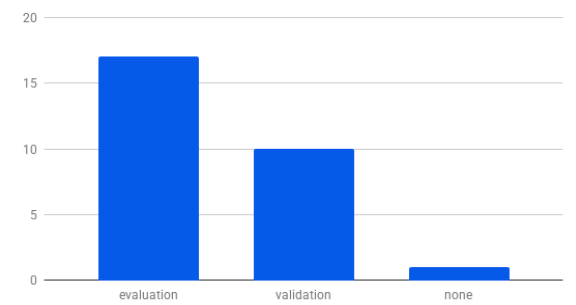


Fig. 9. Analysis methods

vulnerability category arises when an attacker sends invalid data to command an application to perform unintended actions.

Fourth, we identified five vulnerabilities: "weak or unencrypted communications"; "use of broken or risky cryptographic algorithms and clear-text storage"; "information leakage from insecure BLE design"; "insecure device and communication configurations"; and "PII leakage due to access control failures". These vulnerabilities fall into various OWASP Top 10 categories including cryptographic failures, broken access, and security misconfiguration.

Fifth, we identified two additional vulnerabilities: "permission redelegation and leakage due to inadequate encryption", and "MITM attacks triggering vulnerabilities". The two vul-

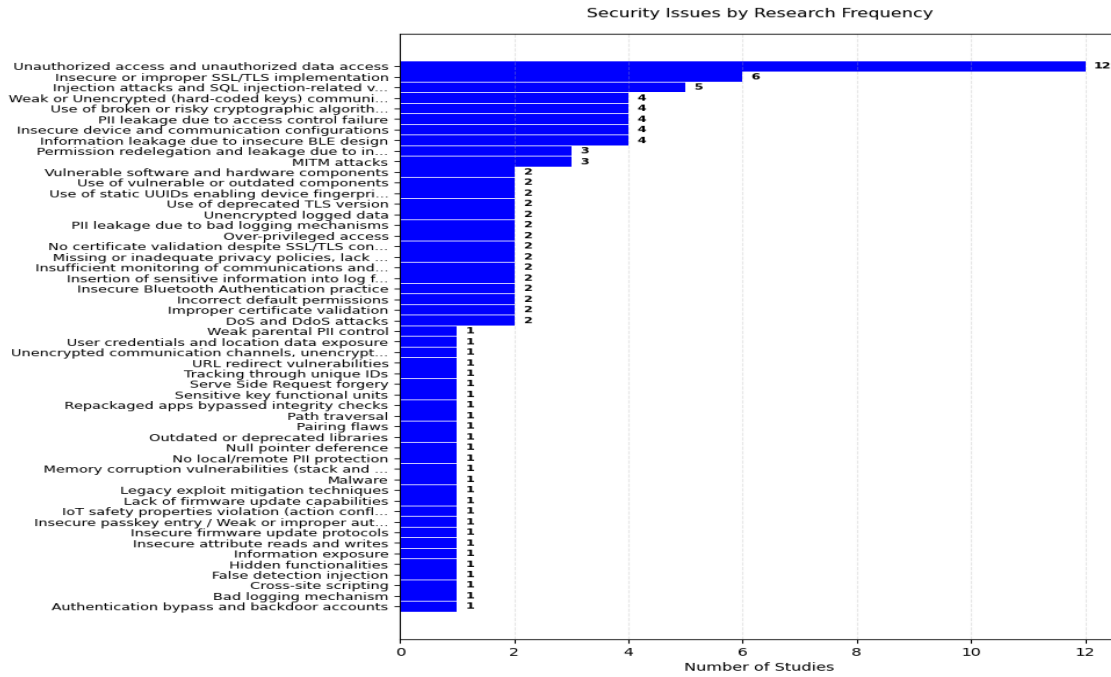


Fig. 10. Critical vulnerabilities classification

nerability types correspond to the OWASP Top 10 categories of cryptographic failures and broken access.

15 distinct vulnerabilities. were reported in two different studies across our selected papers;

27 different vulnerabilities. were reported once in various studies were minor cases identified as security vulnerabilities in IoTMAAs.

Answer to RQ1: What are the most prevalent security vulnerabilities in IoTMAAs based on the IoT threat landscape? As shown in Figure 10, the most critical security vulnerability in IoTMAAs is ‘Unauthorized Access and Data Access,’ detected in 12 of the 39 primary studies.

From this answer, we conclude that vulnerabilities are numerous and present across all layers of IoTMAAs. The answer reveals the danger of the rapid growth of IoT smart devices such as smart locks, smart cameras, or smart intruder alarms that have increasingly become a favorite target for cybercriminals due to their weak security design. In fact, IoT devices often employ weak authentication and authorization mechanisms, making them vulnerable to threats. The most common example involves devices using default passwords, allowing hackers to easily gain access to them without authorization. Our findings align with the growth rate of IoT device manufacturing, which can lead to an implementation rush. We conclude that rapid market adoption tends to prioritize functionality at the expense of security. Therefore, developers will rush IoTMAAs to market, granting permissions they might not fully understand, making unauthorized access the primary attack vector.

To map the detected vulnerabilities to the OWASP 2021 Top 10 and identify the most critical categories, Figure 11

depicts the numbers of each security vulnerability category, and shows the proportion of specific issues in all our PSs. We have included a more detailed table -Table VI- in Appendix A.

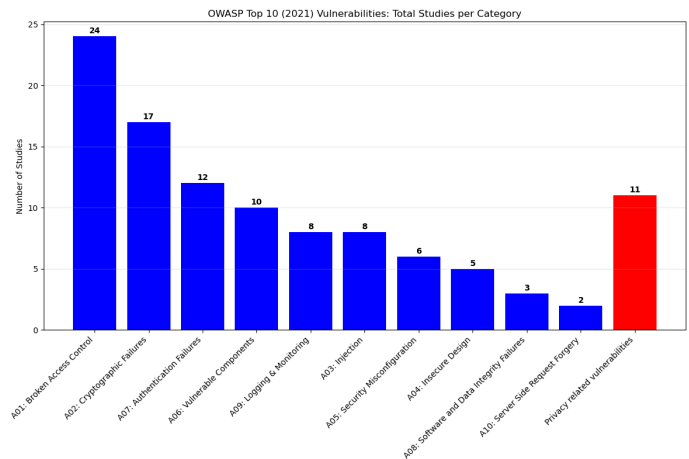


Fig. 11. Critical vulnerabilities categorization with respect to OWASP 2021 Top 10

Note that in Figure 11, the total number of vulnerabilities exceeds the number of PSs, as individual studies often report multiple vulnerabilities. Notably, we identified eleven privacy-related vulnerabilities, such as missing or inadequate privacy policies, lack of encryption of data, action conflicts, and action loops. Interestingly, these privacy issues are not explicitly addressed in the OWASP Top 10 categorization, as shown in Figure 11.

After initially classifying the identified vulnerabilities according to the OWASP 2021 Top 10, we further map these

categories to the eight OWASP MASVS v2.1.0 categories, which are specifically tailored to mobile application security in Table II.

2) *Vulnerability Detection Approaches*: Our analysis of the primary studies shows that a variety of VDAs are employed to detect vulnerabilities in IoTMs. Hence, we grouped the VDAs into seven categories, as shown in Table III. For details, see Table VIII in Appendix A.

Figure 12 shows the distribution of studies across the different VDA categories.

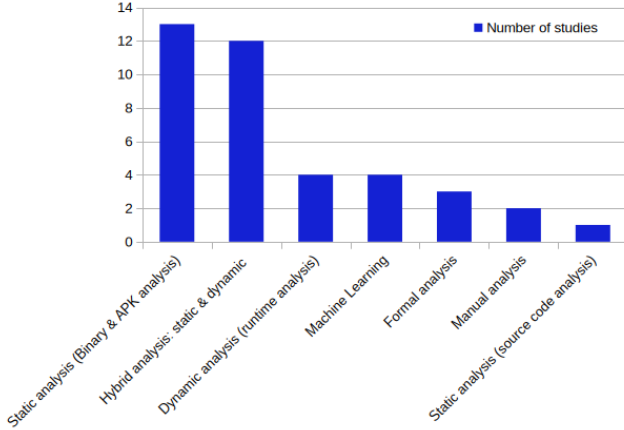


Fig. 12. Vulnerabilities detection approaches distribution across the 39 studies

Here are the main conclusions drawn from Table III:

Static analysis (APK/binary files). is the most used VDA with a total of 13 studies. This involves analyzing compiled Android application packages without executing them, focusing on code structure, permissions, code quality analysis of the decompiled APKs, and configurations. This helps to detect (SQL) malicious injection-related vulnerabilities, privacy policy violations, and cryptographic weaknesses, including the following methods:

- Decompilation and code inspection: converting APK files back to source code to identify security flaws, improper SSL usage, and insecure data handling;
- Permission analysis: extracting and analyzing app permissions to identify potential privilege escalation and unauthorized access vulnerabilities;
- Component analysis: inter-component data flow analysis to detect component hijacking vulnerabilities, permission leakage, and unauthorized data access;
- Automated tool usage: leveraging tools like Mobile Security Framework (MobSF), Ostorlab, and Synk for large-scale vulnerability scanning;
- Reverse engineering: implies manual reverse-engineering of embedded code and its mobile apps to further validate through diverse manual analyzes.

Hybrid analysis approach. is the second most used approach across our selected studies, with 12 studies adopting it. This approach combines static and dynamic analysis to provide broad vulnerability detection using the strengths of both techniques. This process reduces false positives by

validating static findings through dynamic analysis. Hybrid analysis provides complete coverage, enabling the detection of complex vulnerabilities that require both code/ APK or binary analysis and runtime context. Static and dynamic analysis are detailed in the next points.

Dynamic analysis and ML approaches. are the third most used VDAs, with 4 PSs using each approach.

Within ML-based approaches, the authors employed a range of techniques—from general machine learning algorithms to specifically tailored models—aimed at automatically detecting patterns indicative of security vulnerabilities or malicious behavior. These approaches provide key advantages, including scalability to large datasets, automated pattern recognition, and reduced reliance on manual analysis. We distinguish the following applied approaches:

- Random Forest (RF) classifier: classification of network packets containing sensitive information and predicting Android security vulnerabilities using statistical code metrics;
- Deep Learning Malware detection: deep learning sequence classifiers to detect Android malware patterns;
- Feature selection techniques: employing constant removal and recursive feature elimination with SVM, RF, and Decision Tree classifiers to detect injection attacks;
- Evasion attack analysis: systematic analysis of obfuscation and adversarial attack techniques using large-scale datasets.

Within the dynamic analysis approach, the authors analyzed runtime execution and monitored applications to observe actual behavior and identify vulnerabilities through active testing. This method focuses on identifying vulnerabilities caused by memory corruption, privacy leaks, bypassing runtime permissions, and command hijacking. The following methods were used:

- Fuzzing: automatic fuzzing through mobile apps to discover memory corruption vulnerabilities and trigger potential security flaws;
- Traffic interception: real-time collection and analysis of BLE traffic traces from fitness trackers;
- Behavior monitoring: runtime observation of instrumented IoT applications to identify vulnerabilities during execution;
- Attack simulation: command hijacking attacks targeting smart assistants like Alexa, for example.

Formal methods analysis. uses mathematical and logical techniques to model and verify system properties for correctness and security. They were mostly applied to reveal safety and security violations, to verify the correctness and properties of network protocols, and to explore the state space. The following methods were used:

- Model checking: extraction of state models from IRs and application of model checking to verify security properties;
- Deterministic finite automata (DFA): creation of DFA models from smart apps source code to represent expected behavior patterns;

TABLE II
MAPPING OF OWASP 2021 TOP 10 CATEGORIES TO OWASP MASVS v2.1.0 EIGHT CATEGORIES

N°	OWASP Top 10 Categories	OWASP MASVS v2.1.0 Categories	Description
1	A01:2021 Broken Access Control	MASVS-AUTH, MAVS-STORAGE, MASVS-PLATFORM	Restrictions on authenticated users not properly enforced
2	A02:2021 Cryptographic Failures	MASVS-CRYPTO, MASVS-STORAGE, MASVS-NETWORK	Failures related to cryptography leading to exposure of sensitive data.
3	A03:2021 Injection	MASVS-CODE, MASVS-PLATFORM	User-supplied data is not validated, filtered, or sanitized
4	A04:2021 Insecure Design	MASVS-AUTH, MASVS-CODE	Missing or ineffective control design.
5	A05:2021 Security Misconfiguration	MASVS-PLATFORM, MASVS-STORAGE, MASVS-NETWORK	Missing security hardening, or improperly configured permissions
6	A06:2021 Vulnerable and Outdated Components	MASVS-CODE, MASVS-RESILIENCE	Using components with known vulnerabilities
7	A07:2021 Identification and Authentication Failures	MAVS-AUTH, MASVS-STORAGE	Confirmation of user identity, authentication, and session management.
8	A08:2021 Software and data integrity failures	MASVS-CODE, MASVS-RESILIENCE, MASVS-CRYPTO	1
9	A09:2021 Security Logging and Monitoring	MASVS-STORAGE, MASVS-PLATFORM	Insufficient logging, monitoring, and active response.
10	A10:2021 Server-Side Request Forgery	MASVS-NETWORK, MASVS-CODE	Fetching a remote resource without validating user-supplied URL
11	Uncategorised Privacy-related vulnerabilities	MASVS-PRIVACY	Privacy-related vulnerabilities.

TABLE III
CATEGORIZATION OF IOTMAS VULNERABILITIES DETECTION APPROACHES

N°	Primary VDA Category	Studies	Number of papers
1	Static (Binary/APK) analysis	[18], [19], [40], [28], [41], [37], [39], [20], [16], [4], [27], [31], [32]	13
2	Hybrid (static + dynamic) analysis	[42], [35], [29], [15], [30], [43], [44], [26], [22], [45], [17], [24]	12
3	Dynamic analysis	[23], [25], [46], [47]	4
4	Machine Learning	[48], [49], [33], [38]	4
5	Formal analysis methods	[50], [34], [51]	3
6	Manual analysis	[36], [52]	2
7	Static (Source code) analysis	[21]	1

- Alloy analysis: comparison of state-based against rule-based modeling approaches to formally analyze app coordination in IoT platforms;
- Lightweight formal analysis: formal specification extracted from app binaries was used to verify security properties.

Manual methods analysis.: provide a human-based examination combining expert knowledge with systematic investigation techniques. The main advantages of this method are expert involvement in vulnerability detection, context-awareness in security assessment, and the validation of automated findings. The following methods were used:

- Network traffic analysis: manual inspection of network communications, TLS analysis, and protocol behavior examination;
- Code protection analysis: manual analysis of code protection techniques and firmware updating capabilities;
- Security configuration review: manual investigation of abnormal inputs, web server security issues, and privacy

policy compliance.

Static analysis (source code). examines app source code directly to identify security vulnerabilities through code inspection and pattern matching. It focuses on areas such as memory corruption vulnerabilities, insecure data transmission and storage, authentication bypass mechanisms, and protocol implementation flaws. The following methods were used:

- Code metrics analysis: statistical analysis of code patterns and metrics to predict security vulnerabilities;
- Privilege separation analysis: source code examination for separation issues and weaknesses in protecting sensitive data;
- IoT code translation: converting IoT source code into intermediate representations (IR) for systematic security analysis;
- Manual code review: direct inspection of companion app source code and configuration files.

Answer to RQ2: Which methods are used to detect vulnerabilities in IoTMAS?

Our analysis of the primary studies identified seven VDAs currently in use: static analysis of APK/binary files, hybrid analysis, static source code analysis, dynamic analysis, machine learning-based analysis, formal methods, and manual analysis.

To address this question, we identified all VDAs reported in the selected literature, quantified their frequency of use, and ranked them from most to least utilized across our primary studies.

The prevalence of static analysis as a frequently used VDA suggests that a significant proportion of security vulnerabilities in IoTMAS are associated with APK and binary files, which suggests that secure coding practices are not always followed when developing these apps. This conclusion is aligned with the results reported by Allen et al. [32]. The second most

prevalent approach is hybrid analysis, which combines IoTMA binary file analysis with runtime analysis, while static analysis of the app binaries is also used as a standalone method. Hybrid analysis is generally more effective at detecting security vulnerabilities because it combines both static and dynamic approaches, increasing the potential to identify a great number of issues, albeit with higher effort. Note that the effectiveness of static analysis can vary depending on the IoTMA, the device, and its operating system, resulting in different numbers of detected vulnerabilities. Finally, we observe a substantial gap between the first two most utilised VDAs and the third-ranked approach, suggesting that other approaches may be considered supplementary to the first two.

In addition to the prevalence of the two aforementioned VDAs, our analysis reveals that ML represents a promising alternative when datasets are readily available. This was demonstrated by [33], where the authors utilised the well-known AWID dataset, performed a customised feature selection, and achieved a detection rate of 99%. In fact, deep learning algorithms emerge as particularly noteworthy VDAs for IoTMAs because of their capacity to scale analysis across large datasets while considerably reducing manual intervention.

3) *Effectiveness of VDAs in detecting vulnerabilities:* Before evaluating VDA effectiveness, we present the distribution of VDAs and detected vulnerabilities in Figure 13. In Table IV, we provide details for each VDA, including the detected vulnerabilities and the datasets used for evaluation. Finally, we conclude with the answer to RQ3.

Figure 13 shows the distribution of VDAs used to detect the most prevalent security vulnerabilities in IoTMAs, as identified in RQ1. Hybrid approaches exhibit the highest detection capability, identifying nine distinct vulnerability types, followed by static analysis of binary files with seven. Dynamic analysis ranks third with four detected vulnerabilities, while static source code analysis and machine learning-based analysis each detected three. Finally, formal methods and manual analysis identified two and one vulnerability types, respectively.

Beyond quantifying the vulnerabilities detected by each VDA, we assess the methodological rigor of each approach before evaluating its effectiveness. To this end, we examined the validation methodologies used in each study to establish confidence in the findings presented in Figure 13. Our analysis focuses on two key dimensions: the evaluation metrics used to validate each approach, and the characteristics of the dataset employed for metric assessment. The results of this validation analysis are summarized in Table IV.

For clarity, the metrics were grouped into categories and summarized in Table IV. For more details on metrics and datasets, see Table XII in Appendix A.

In this analysis, we systematically examine each of the most prevalent vulnerabilities identified in RQ1, along with the metrics used for validation and the datasets on which the VDAs were tested.

Based on this analysis, we address RQ3 and provide key insights into the effectiveness of each VDA, either in comparison with other approaches or with respect to the specific vulnerabilities each can detect.

Unauthorized access and unauthorized data access.

The metrics used to validate VDAs for this vulnerability category primarily rely on empirical measurements, quantitative assessments of vulnerability prevalence, and real-world exploit tests. Four VDAs were able to detect this type of vulnerability: hybrid analysis, static APK/binary files analysis, dynamic analysis, and static source code analysis. Below is the distribution of the metrics:

- Empirical validation metrics: focus on real-world demonstration of attacks and quantifiable outputs from tested applications. This includes attacks success rate, where authors of [16], [17], [18], [20], [21] evaluated how often detected issues led to successful exploitation, and [17], [18] measured impact of executing real-world attacks; the total counts of detected unauthorized access flaws in app datasets, referred to as vulnerability discovery counts, were reported by the authors of [15], [16], [18], and [20]. Responsible disclosure to vendors, as demonstrated by [16] and [24], involved reporting bugs to companies such as Fitbit. The subsequent patches confirmed the reported issues. In addition, real-world exploitability was demonstrated by [15], [17], and [24], showing that the detected vulnerabilities could compromise devices or data.
- Quantitative assessment reports numerical measures of specific properties of analyzed apps or devices. This includes large-scale detection coverage, where [18] and [22] validated their detection approaches on large datasets, measuring the number of vulnerable apps: 431 out of 18,166 in [18] and over 10,000 out of 17,243 in [22]. Property violation counts were reported by [20], who identified 17 of 20 unique property violations across 17 tested apps. Finally, prevalence rate was highlighted by [21], showing that 70% of tested apps exhibited at least one of six vulnerability types.
- Technical performance and accuracy validation: this category of metrics validates how precise and efficient detection approaches are in identifying unauthorized access vulnerabilities. They include True Positive/True Negative Rate (sensitivity and recall). For instance, the authors of [19] reported a TPR of 81% against the 254 apps flagged as potentially vulnerable, with a TNR of 19%. The authors of [23] demonstrated the efficiency of their approach with a classifier accuracy of 97.6%. Similarly, [20] achieved a detection rate accuracy of 100% regarding unauthorized access vulnerabilities. Finally, the authors of [19] went further in validation by measuring the runtime and performance overhead of their tool, reporting a median execution time of 37.02 seconds, confirming its scalability, and showing satisfactory resource usage.
- Qualitative validation: presents VDA validation methodologies. They include proof-of-concept demonstrations. For example, the authors of [24], [17], and [21] showed unauthorized access either by reverse-engineering the

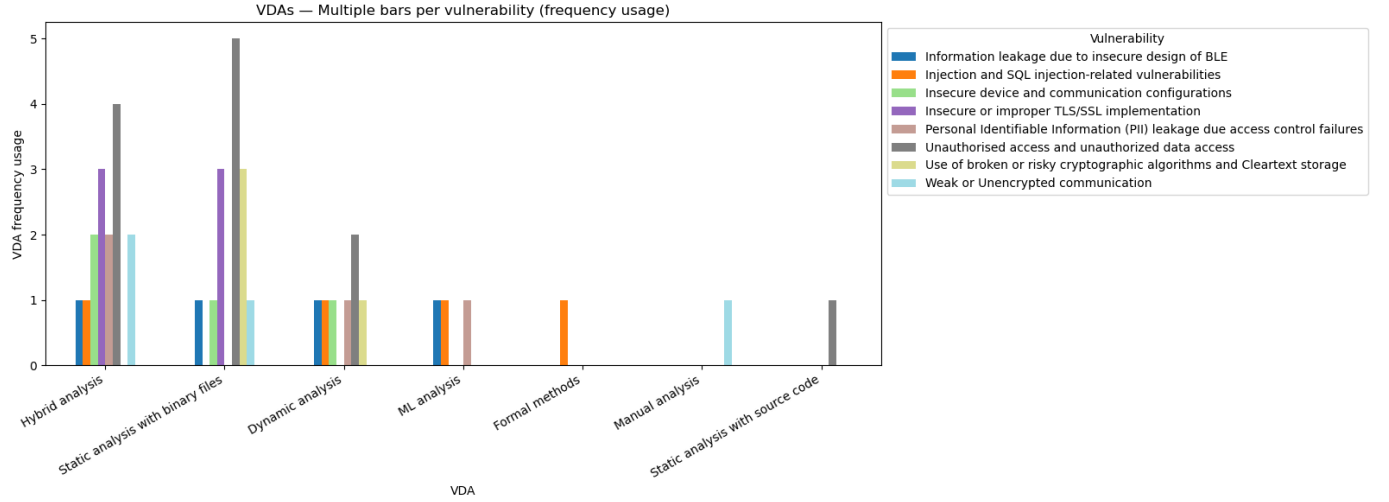


Fig. 13. VDAs used across the 39 PSs for vulnerability detection

TABLE IV
VDAs' EFFECTIVENESS CRITERIA

N° VDAs	Vulnerability types count	Categories of validation metrics	Dataset sources
1 Hybrid analysis [42], [35], [29], [15], [30], [43], [44], [26], [22], [45], [17], [24]	7	Empirical validation, quantitative evaluation, qualitative validation, responsible disclosure of vulnerabilities to vendors and vendors acknowledgment	Amazon, Google PlayStore, SmartThings Github repos, Fitbit trackers.
2 Static analysis with APK/binary files [18], [19], [40], [28], [41], [37], [39], [20], [16], [4], [27], [31], [32]	6	Empirical validation, quantitative evaluation, technical performance, and accuracy validation, comparison with existing tools, analysis execution time per app, manual case studies demonstration	Google Play Store, Amazon, Google personal and business account, mHealth apps market
3 Dynamic analysis [23], [25], [46], [47]	6	Empirical validation	Fitbit data traces collected from a gymnasium, Play Store.
4 Machine Learning [48], [49], [33], [38]	3	Technical performance, and accuracy validation, comparative analysis	AWID, network traffic collected from IoT companion apps
5 Formal methods analysis [50], [34], [51]	2	Empirical validation, technical performance, comparison with existing tools	F-Droid, Mal-Genome, Debrin, Androzoo.
6 Static analysis with source code [21]	2	Qualitative validation	IoT device trackers data
7 Manual analysis [36], [52]	1	Empirical validation, qualitative validation	Smart Toys data

mobile application firmware or by launching attacks on real devices. They also include manual confirmation of findings. The authors of [22] and [24] validated vulnerabilities by decompiling apps, inspecting network flaws, or testing device behavior. Finally, they include case study implementations. The authors of [22] and [24] presented detailed examples of real apps and devices that exhibited unauthorized access vulnerabilities.

Insecure or improper TLS/SSL implementation.: the most prevalent technique used to validate this vulnerability is attack success rate measurement, appearing in 3 studies, complemented by test case-based validation for systematic SSL/TLS vulnerability detection.

- Attack success rate: reports the measurements of direct success rate of attacks [31], the attack success rate with reproducibility validation [44], and empirical evaluation demonstrating how flawed TLS configuration enables

effective exploitation [27].

- Test case-based validation: where the authors of [30] employed test cases to check for common TLS/SSL vulnerabilities by evaluating how apps were handling self-signed or expired certificates used to simulate Man-In-The-Middle attacks.
- Coverage and detection rate: measures how thoroughly the VDA scans the app? and TLS/SSL usage, and how many vulnerabilities it detects. In [29], authors evaluated how many TLS-related API were analyzed by the VDA and the percentage of vulnerability instances found.
- Reproducibility of attacks: demonstrating that vulnerabilities can be reliably triggered under similar conditions. The authors of [44] demonstrated that attacks on vulnerable apps consistently produced the same exploitation behavior. The authors of [27] also demonstrated how flawed TLS configurations enabled effective exploitation

of the apps.

SQL injection-related vulnerabilities.: validation metrics are multifaceted here, but the predominant aspect is related to attacks as seen in [26] and [25]. Other studies applied standard ML metrics—such as accuracy, TP, TN, FP, and FN—using real-world smart devices and mostly publicly available datasets.

Personal Identifiable Information (PII) leakage due access control failures.: the validation processes in these studies were largely practical, including test case-based validation and proof-of-concept attacks, although standard ML metrics were also applied to real device data.

Use of broken or risky cryptographic algorithms and clear text storage of sensitive information.: The most dominant technique used to validate the VDA detecting this vulnerability is the attack success rate [26], [4]. Another approach is manual code inspection by a SecDevOps developer with 10 years of experience [32]. Finally, validation also includes the demonstration of attack success [28].

Information leakage due to insecure BLE design. The metrics used are diverse. [38] reported accuracy greater than 99% and an F1-score above 0.99, while [23] reported 89% accuracy with a false positive rate below 5%. [24] demonstrated practical exploitability through proof-of-concept attacks. Other validation approaches include reproducibility, where attacks and exploits were reliably reproduced and demonstrated. They also include manual verification through reverse engineering and protocol analysis, as well as responsible disclosure to vendors, with acknowledgment of the vulnerabilities. For example, the authors of [37] validated their VDA by conducting a large-scale measurement on 11,045 BLE apps, identifying a prevalence of 81% of vulnerabilities. They further used real-world case studies to demonstrate the exploitation of vulnerabilities in actual devices running older versions.

Weak or Unencrypted communication.: The authors used various validation techniques to assess their VDA. In [27], the authors used empirical techniques to demonstrate the feasibility of exploiting vulnerabilities within a Belkin WeMo ecosystem. [17] measured attack success rates and real-world exploitability, while [35] and [36] employed practical cases, using network analysis and manual verification to demonstrate the effectiveness of their approaches.

Insecure device and communication configuration.: validation approaches include measuring vulnerability prevalence rates, performing manual analysis and verification, conducting case studies and impact assessments, analyzing firmware via the companion app, launching live device attacks, and evaluating false positives [35], [22], [39].

Based on our analysis of the evaluation metrics and datasets used by the VDAs, we conclude that, although each VDA has distinct strengths and weaknesses, all approaches successfully identified their targeted security vulnerabilities.

Following the analysis of metrics and datasets used to validate VDAs, we now examine how the VDAs performed in terms of vulnerability detection effectiveness comparatively:

Hybrid VDAs. demonstrated greater performance across all categories of vulnerabilities. Hybrid approaches consistently appear as the primary or co-primary detection method for

seven of the eight most prevalent vulnerabilities, as shown in Figure 13, and serve as the main VDA for nearly all listed vulnerabilities. By combining multiple detection techniques, they achieve higher accuracy, with validation metrics indicating strong effectiveness. For instance, hybrid analysis excels at detecting runtime behaviors such as TLS misconfigurations and PII leakage that static analysis alone might miss. We observe that [30], [29], and [24] successfully identified runtime-specific issues such as improper SSL/TLS configurations; however, vulnerabilities hidden in obfuscated code may still be missed.

Static analysis (binary and APK). comes next as one of the most effective VDAs, used to detect six of the eight most prevalent vulnerabilities. It was validated using large datasets (18,166 and 17,243 apps). We observed that it is particularly effective at identifying code/APK-level issues such as permissions, hardcoded keys, or outdated libraries, as it is shown in [28] and [39]. Tools including CHEX [19] and Amandroid[40] demonstrated success in detecting component hijacking and data leaks. This VDA was generally proven to be very effective in revealing cryptographic and permission-related issues, as shown in [4].

Dynamic analysis. was proven to be effective on specific vulnerability types such as protocol related issues, and was used to detect six among the eight most dominant vulnerabilities. It also successfully detected runtime exploits such as memory corruption in [46], or injection attacks in [25]. Additionally, tools such as IOTFuzzer in [46] and DIANE in [15] were successful in fuzzing IoT devices. However, this analysis approach requires environment setup and is prone to missing vulnerabilities in case others are not triggered during testing.

Machine Learning. has shown great potential in detecting three of the eight most prevalent vulnerabilities such as privacy-related vulnerabilities. This approach showed promising results with high accuracy and F1-scores. For example, [38] effectively classified encrypted traffic, and [50] detected malware. However, its effectiveness heavily depends on the quality of the training data; poor-quality data can significantly reduce performance.

Formal analysis approach. was used once to detect security misconfiguration vulnerabilities, such as in [34].

Manual analysis. showed high effectiveness in detecting design flaws like weak parental control and tracking issues as seen in [36]. However, this approach is highly prone to human errors and is not scalable.

Static analysis with source code. detected three among the eight most prevalent vulnerabilities identified in RQ1. It was validated using strong practical metrics, tested on real-world datasets such as data extracted from smart camera apps. This VDA has limited applicability due to the scarcity of available app source code. However it is effective in detecting implementation-level vulnerabilities such as TLS/SSL flaws or weak communications, as seen in [27], where authors detected TLS misconfigurations. This approach is also ineffective at detecting runtime or network-level vulnerabilities, which are prevalent in IoT ecosystems.

Answer to RQ3: Which validation metrics and datasets are used in VDAs for IoTMA, and how effective are they in assessing vulnerability detection?

Our analysis identified a range of metrics and numerous distinct datasets. We classified the metrics into five categories: empirical, quantitative assessment, technical performance and accuracy validation, qualitative, and comparative assessment. In contrast, the datasets could not be categorized in the same way because of their diversity and their specificity to particular IoT devices. Most of these datasets, however, were sourced from application stores such as Google, Amazon, and Huawei.

Through metric validation, our analysis revealed that our VDAs were effective in detecting the specific vulnerabilities they were designed to target.

We observed that to detect runtime-level vulnerabilities, hybrid analysis, dynamic analysis, and formal analysis were proven very effective. For code-level or implementation-level vulnerabilities, static source code analysis has been proven to be the best. For configuration-level vulnerabilities, static analysis of APK/binary files yielded better results. Machine learning was effective in detecting privacy-related issues, while manual analysis demonstrated high effectiveness in detecting design flaws.

4) *Recommendations to guide researchers and practitioners in improving security in IoTMAs:* The analysis of our PSs led to a deep understanding of the current landscape of security vulnerabilities of IoTMAs, sometimes associated with a specific smart device. We identified the most prevalent vulnerabilities in these apps, along with the VDAs used to detect them, which we have ranked in terms of effectiveness, based on the validation metrics and datasets used to test each VDA. These metrics were used to assess the effectiveness of each VDA. Based on these results, we propose recommendations for best practices in developing secure IoTMAs. First, we provide general recommendations to researchers and practitioners, followed by specific recommendations tailored to each group.

- 1) **A combination of hybrid analysis, static binary/APK analysis, dynamic analysis, and static source code analysis** yields ensure the highest vulnerability detection coverage. As demonstrated in our study, each VDA has distinct advantages, and their combination enhances vulnerability detection. This approach is recommended because it covers both source code and APK analysis—revealing issues such as excessive permissions, hardcoded keys, and outdated libraries—as well as runtime vulnerabilities, including TLS misconfigurations and PII leakage. Runtime exploits are detected by fuzzing the IoT device on which these apps are running.
- 2) Performing **reverse engineering** during static analysis reveals IoTMAs flaws hidden within application binaries.
- 3) **ML analysis approach**, primarily through supervised learning algorithms, is a promising approach to automate vulnerability detection, and to reduce human

effort, which is inherently prone to errors. It enables scalable analysis beyond manual inspection. However, the success of ML-based vulnerability detection heavily depends on the quality and the accuracy of labeled training data. It is recommended to perform validation analysis to identify True Positives (TP), True Negatives (TN), False Positives (FP), and False Negative (FN) among the classified outputs. ML can improve their classification accuracy over time, but only when trained on properly trained and validated datasets.

- 4) Given its high cost and error proneness, **manual analysis** is not recommended for IoTMA.
- 5) We emphasize on the need to intensify studies on securing IoTMAs against unauthorized access vulnerabilities, as they remain the most prevalent to date. Such vulnerabilities expose systems to attacks including man-in-the-middle (MITM) and denial of service (DoS) attacks.

Answer to RQ4: What recommendations can be formulated to guide researchers and practitioners in improving security in IoTMAs based on the findings of RQ3?

We formulated two specific recommendations for researchers and four for practitioners.

For researchers.

To detect security vulnerabilities in IoTMAs, we recommend that researchers follow these steps:

- Prioritize a multi-stage pipeline combining 1) static analysis tools to detect hardcoded keys, weak TLS, improper permissions-like vulnerabilities, 2) dynamic monitoring, to detect MITM/weak encryption, unauthorized access-like vulnerabilities, and 3) manual inspection. To increase efficacy, manual inspection should be optimized with ML algorithms to enhance detection and reduce overhead, particularly for BLE leak and PII exposure vulnerabilities.
- We recommend designing a standardized effectiveness metrics suite to establish common benchmarks for comparing VDA performance, as the metrics used in the primary studies were inconsistent. This benchmark should include testing VDAs on diverse, real-world datasets such as Androzoo, Google Play, or other popular IoTMA marketplaces with a representative number of applications. We also encourage researchers to engage in responsible disclosure and obtain vendor acknowledgment; to validate the real-world impact of their approaches, and we urge them to release their tools and datasets to support reproducibility purposes.

For practitioners. Similarly, the following steps are recommended for practitioners:

- Integrate VDAs into the software Development Life Cycle (SDLC) of IoTMAs by incorporating recommended static and dynamic analysis techniques into CI/CD pipelines. This can be achieved by reading researchers' feedback on specific vulnerabilities detected on

smart devices. They can also perform risk-based scanning of permissions, cryptographic operations, and data flows before releasing applications.

- Place greater emphasis on providing more robust mechanisms to secure BLE, implement TLS properly, and enforce access control while developing IoTMAAs. Practitioners could for example check for runtime leaks and perform manual traffic inspection.
- Design standardized effectiveness metrics for IoT smart devices and applications that go beyond simple vulnerability counts.
- Foster partnerships between academia and industry to enable researchers to access up-to-date apps. In exchange, practitioners could request responsible disclosure of vulnerabilities detected by researchers, provide appropriate acknowledgment, and release patches immediately after addressing the reported issues.

VI. DISCUSSION AND FUTURE RESEARCH CHALLENGES

A. Discussion of Results

This section presents the results of our systematic literature review of IoT security vulnerabilities in IoTMAAs. We discuss the findings based on the analysis categories that we used to present our results,

Vulnerability issues. In our study, we map the most prevalent vulnerabilities to the OWASP 2021 Top 10 classification [53]. Table VI shows that the most vulnerable category is ‘Broken Access,’ followed by vulnerabilities related to ‘Cryptographic Failures,’ ‘Authentication Failures,’ and privacy. The complete mapping is shown in Figure 11.

Based on the mapping presented in Table II, six of the eight OWASP MASVS v2.1.0 categories—AUTH, STORAGE, PLATFORM, CRYPTO, NETWORK, and PRIVACY—are found to be among the most vulnerable categories. This result highlights the need to strengthen IoTMAAs security. This is consistent with the OWASP MASVS project goal, which underscores that secure mobile applications require thorough coverage across all MASVS categories, as weaknesses in any one of them can compromise the overall security posture.

Industry involvement. Out of the 39 studied papers, only one involved industry. However, five studies -[23], [27], [39], [51], [47]- responsibly disclosed detected vulnerabilities to vendors. For example, authors of [47] reported the three new attacks found in the intent-matching process of Amazon Alexa platform to Amazon, enabling the company to prevent security threats when developing mobile applications. This approach highlights a concerning bidirectional gap in IoT security research between academia and industry. Do academic researchers often develop their research without consulting industry-reported security vulnerabilities, and potentially pursue theoretical vulnerabilities? Simultaneously, when researchers identify vulnerabilities, do industry manufacturers overlook their findings and decide not to act on them? We advocate for a more symbiotic collaboration between IoT security researchers and industry leaders through formal vulnerability disclosure processes, joint research initiatives, and regular knowledge exchange forums. This would allow

academics to ground their research in practical industry needs while giving manufacturers access to specialized security expertise before vulnerabilities are exploited. Establishing such collaborative frameworks would greatly improve the security landscape of IoTMAAs by ensuring that research addresses relevant threats from findings.

Platforms. Our analysis revealed a significant imbalance in the platforms involved in examining IoT mobile application security vulnerabilities. Out of 39 studies, 30 (76.9%) focused exclusively on Android platforms, while only one study (2.6%) examined iOS applications. The remaining studies were distributed across SmartThings (two studies, 5.1%), Amazon Alexa (one study, 2.6%), and generic IoT platforms (five studies, 12.8%), as shown in Figure 4. However, this distribution should not be interpreted as evidence that Android platforms inherently harbor more security vulnerabilities than iOS or other platforms. Rather, it likely reflects Android’s open-source nature and greater accessibility to researchers, compared to iOS’s closed ecosystem and stricter developer constraints. These accessibility differences create a methodological bias in IoT security research that must be acknowledged when drawing conclusions. We recommend that future security assessments explicitly account for these platform accessibility disparities and develop innovative methodologies to evaluate closed OSs. Additionally, industry-academia collaborations could potentially provide researchers with greater access to proprietary platforms, enabling more balanced security analyzes across the entire IoTMA landscape.

Comparison of our study research with related studies.

Due to differences in research objectives and focus areas among our study and existing related works as shown in Table V, we found it challenging to directly compare them. Actually, while Svacina et al. focused on log analysis techniques across various software systems, Reddy et al. examined Swarm Intelligence algorithms for IDS optimisation, and Alshamsi et al. investigated ML-based malware detection in smart home networks, our study targets vulnerability detection approaches in IoT mobile applications.

TABLE V
RESEARCH OBJECTIVES AND FOCUS OF RELATED STUDIES AND OUR WORK

Study	IoT Layer	Focus Area	Detection Target	IoTMAAs Coverage
Svatina et al.	Infrastructure	Log analysis	System intrusions	Indirect (infra. only)
Reddy et al.	Network	IDS optimization	Network intrusions	Not addressed
Alshamsi et al.	Device and network	Malware detection	Smart home threats	Not addressed
Our study	Applications	Vulnerability detection	Mobile app vulnerabilities	Primary focus

B. Threats to Validity

We have organized the threats to the validity of this study on four aspects: construct validity, internal and external validity, and conclusion validity.

- **Construct validity.** This threat has to do with the data collection process and the sources we investigated. It concerns the process of selecting the PSs, and how we conducted data extraction from our PSs with respect to our RQs.
 - Missing relevant studies. Incompleteness is a threat. Our search relied on one query. The accuracy of our search depends on the effectiveness of digital library search engines in retrieving papers relevant to the query. To mitigate this threat, we used EV, a comprehensive engineering database that includes journals, conference proceedings, dissertations, technical standards, trade magazines, and technical reports information from reputable digital libraries [54]. We conducted three rounds of snowballing to identify additional relevant studies that may have been missed in our initial search.
 - PS selection bias. During the selection of PSs, we may have excluded relevant studies. The subjective nature of manually conducting the selection process could contribute to this biased selection. To mitigate this selection bias, we defined the purpose of the study and the research questions in advance, following PRISMA guidelines, and established clear inclusion and exclusion criteria.
 - Data extraction bias. The data extraction process involved extensive manual efforts, which may be subject to personal bias. To minimize this bias, we defined the data collection protocol in the form of an online spreadsheet, including all the required information. The four authors followed the agreed protocol and independently conducted the data extraction. We used interrater agreement and several discussion sessions involving all authors to reach a common agreement. Another threat arises from the inconsistent terminology used in the PSs. We discussed each of these inconsistencies and agreed on a unified vocabulary.
- **Internal validity.** Internal validity concerns the methods used in the study and related conclusions.
 - Scope of the review. While the research questions may not encompass all aspects of VDAs for IoTMAs or all existing VDAs, we provide the most comprehensive analysis possible, informed by a current security practitioner among the authors to complement academic and industry perspectives.
 - Method of the review. The methodology to conduct research may introduce various threats, including the potential for bias, which can undermine the reliability and validity of the findings. To mitigate this threat, we followed the updated PRISMA guidelines and carefully selected relevant studies while removing irrelevant ones. All authors participated in defining and reviewing the inclusion/exclusion criteria.
- **External validity.** External validity refers to the extent to which our findings can be generalized to the detection

of security vulnerabilities in IoTMAs. Two main aspects of this research may hinder its external validity.

- This review focuses exclusively on academic research. Industry practices may not be included if they were not reported in academic publications. We reviewed studies published within a specific time frame, which may limit the extent to which the findings can be generalized. However, our findings are valuable for both academia and industry practitioners.
 - Our primary studies may not cover types of IoTMAs; for instance, we did not find studies focusing on industrial IoT, smart energy, or smart cities. While these ecosystems may exhibit characteristics distinct from those studied, they also share common features inherent to IoTMAs and the broader IoT ecosystem.
- **Conclusion validity.** concerns the degree to which the conclusions drawn from the data extracted are reasonable and valid. We ensured the validity of our conclusions by carefully analyzing the data extracted from PSs. To enhance validity, we conducted multiple discussion sessions wherein we collectively drew and cross-verified our conclusions against the extracted data.

VII. CONCLUSION

In this study, our objectives were to: (1) identify, classify, and prioritize critical security vulnerabilities in IoTMAs; (2) survey and analyze existing VDAs used to detect security vulnerabilities in IoTMAs; (3) critically evaluate the effectiveness of these VDAs; and (4) provide evidence-based recommendations derived from the limitations of current detection approaches. To achieve this, we conducted a systematic literature review of selected primary studies, reviewing 39 out of 856 papers retrieved from four academic databases. Our analysis identified 52 security vulnerabilities in IoTMAs, nine of which are critical. This highlights a considerably expanded and multifaceted security threat surface, which renders IoTMAs highly susceptible to exploitation through a wide range of attack vectors, such as unauthorized access and SQL injection vulnerabilities, making them vulnerable to exploitation. We uncovered seven VDAs used to detect these vulnerabilities, showing that no single method can address all vulnerabilities, so using a combination of approaches is likely needed to effectively secure IoTMAs. Our analysis of VDA effectiveness, based on relevant metrics and datasets, found that studies employ context-specific evaluation methods tailored to particular vulnerabilities and detection techniques. This lack of standardization makes cross-study comparisons difficult. Finally, we provided evidence-based recommendations to guide researchers and practitioners in developing and assessing security in IoTMAs.

Based on our findings, we identify three major research directions essential for advancing the field. First, establishing standardized evaluation metrics to address disparities in VDA assessment. Second, further investigating the capabilities of ML-based detection approaches to more efficiently target key vulnerabilities. Third, collecting, curating, and publishing

datasets that represent a diverse range of IoT devices. In the short term, our work will focus on developing these standardized metrics and constructing a benchmark IoTMA dataset for popular devices, ensuring that both datasets and validation sets comprehensively cover each major category of security vulnerability.

Replication Package

The replication package is openly available at ³, or using the following DOI: 10.5281/zenodo.17095200.

REFERENCES

- [1] J. Pyfer, "Internet of things — definition, history examples, and privacy concerns," <https://www.britannica.com/science/Internet-of-Things>, accessed: 2025-05-13.
- [2] statistica, "Forecast enterprise spending on internet of things (iot) worldwide from 2022 to 2030 (in billion us dollars)," <https://www.statista.com/statistics/1369033/global-iot-market-size/#:~:text=Global%20IoT%20enterprise%20market%20worldwide%202022%2D2030&text=The%20global%20market%20for%20enterprise,by%20the%20end%20of%202030.,> accessed: 2025-03-20.
- [3] D. Marikyan, S. Papagiannidis, and E. Alamanos, "A systematic review of the smart home literature: A user perspective," *Technological Forecasting and Social Change*, vol. 138, pp. 139 – 154, 2019. [Online]. Available: <http://www.sciencedirect.com/science/article/pii/S0040162517315676>
- [4] A. Brilingaitė, L. Bukauskas, and E. Kutka, "Detection of premeditated security vulnerabilities in mobile applications," in *ECCWS 2019 18th European Conference on Cyber Warfare and Security*. Academic Conferences and publishing limited, 2019, p. 63.
- [5] D. K. K. Reddy, J. Nayak, H. Behera, V. Shanmuganathan, W. Viriyasitavat, and G. Dhiman, "A systematic literature review on swarm intelligence based intrusion detection system: past, present and future," *Archives of Computational Methods in Engineering*, vol. 31, no. 5, pp. 2717–2784, 2024.
- [6] O. Alshamsi, K. Shaalan, and U. Butt, "Towards securing smart homes: A systematic literature review of malware detection techniques and recommended prevention approach," *Information*, vol. 15, no. 10, p. 631, 2024.
- [7] J. Svacina, J. Raffety, C. Woodahl, B. Stone, T. Cerny, M. Bures, D. Shin, K. Frajtak, and P. Tisnovsky, "On vulnerability and security log analysis: A systematic literature review on recent trends," in *Proceedings of the International Conference on Research in Adaptive and Convergent Systems*, 2020, pp. 175–180.
- [8] OWASP Foundation, "OWASP iot ten 10," <https://owasp.org/www-project-top-ten/>, accessed: 2025-10-29.
- [9] —, "OWASP mobile application security," <https://owasp.org/www-project-mobile-app-security/>, accessed: 2025-10-29.
- [10] E. Schiller, A. Aidoo, J. Fuhrer, J. Stahl, M. Zörjen, and B. Stiller, "Landscape of iot security," *Computer Science Review*, vol. 44, p. 100467, 2022.
- [11] D. El-Masri, F. Petrillo, Y.-G. Guéhenec, A. Hamou-Lhadj, and A. Bouziane, "A systematic literature review on automated log abstraction techniques," *Information and Software Technology*, vol. 122, p. 106276, 2020. [Online]. Available: <http://www.sciencedirect.com/science/article/pii/S0950584920300264>
- [12] B. Kitchenham, O. Pearl Brereton, D. Budgen, M. Turner, J. Bailey, and S. Linkman, "Systematic literature reviews in software engineering - a systematic literature review," *Inf. Softw. Technol.*, vol. 51, no. 1, p. 7–15, Jan. 2009. [Online]. Available: <https://doi.org/10.1016/j.infsof.2008.09.009>
- [13] B. Kitchenham, "Procedures for performing systematic reviews," *Keele, UK, Keele Univ.*, vol. 33, 08 2004.
- [14] PRISMA, "Prisma statement," <https://www.prisma-statement.org/>, accessed: 2024-11-30.
- [15] N. Redini, A. Continella, D. Das, G. De Pasquale, N. Spahn, A. Machiry, A. Bianchi, C. Kruegel, and G. Vigna, "Diane: Identifying fuzzing triggers in apps to generate under-constrained inputs for iot devices," in *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2021, pp. 484–500.
- [16] W. Zhou, Y. Jia, Y. Yao, L. Zhu, L. Guan, Y. Mao, P. Liu, and Y. Zhang, "Discovering and understanding the security hazards in the interactions between {IoT} devices, mobile apps, and clouds on smart home platforms," in *28th USENIX security symposium (USENIX security 19)*, 2019, pp. 1133–1150.
- [17] Q. Zhang and Z. Liang, "Security analysis of bluetooth low energy based smart wristbands," in *2017 2nd International Conference on Frontiers of Sensors Technologies (ICFST)*. IEEE, 2017, pp. 421–425.
- [18] C. Zuo, H. Wen, Z. Lin, and Y. Zhang, "Automatic fingerprinting of vulnerable ble iot devices with static uuids from mobile apps," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 1469–1483.
- [19] L. Lu, Z. Li, Z. Wu, W. Lee, and G. Jiang, "Chex: statically vetting android apps for component hijacking vulnerabilities," in *Proceedings of the 2012 ACM conference on Computer and communications security*, 2012, pp. 229–240.
- [20] Z. B. Celik, P. McDaniel, and G. Tan, "Soteria: Automated {IoT} safety and security analysis," in *2018 USENIX annual technical conference (USENIX ATC 18)*, 2018, pp. 147–158.
- [21] E. Fernandes, J. Jung, and A. Prakash, "Security analysis of emerging smart home applications," in *2016 IEEE symposium on security and privacy (SP)*. IEEE, 2016, pp. 636–654.
- [22] P. Sivakumaran, C. Zuo, Z. Lin, and J. Blasco, "Uncovering vulnerabilities of bluetooth low energy iot from companion mobile apps with ble-guide," in *Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security*, 2023, pp. 1004–1015.
- [23] A. K. Das, P. H. Pathak, C.-N. Chuah, and P. Mohapatra, "Uncovering privacy leakage in ble network traffic of wearable fitness trackers," in *Proceedings of the 17th international workshop on mobile computing systems and applications*, 2016, pp. 99–104.
- [24] J. Classen, D. Wegemer, P. Patras, T. Spink, and M. Hollick, "Anatomy of a vulnerable fitness tracking system: Dissecting the fitbit cloud, app, and firmware," *Proceedings of the ACM on interactive, mobile, wearable and ubiquitous technologies*, vol. 2, no. 1, pp. 1–24, 2018.
- [25] Y. Lee, S. Woo, J. Lee, Y. Song, H. Moon, and D. H. Lee, "Enhanced android app-repackaging attack on in-vehicle network," *Wireless Communications and Mobile Computing*, vol. 2019, no. 1, p. 5650245, 2019.
- [26] E. Chatzoglou, G. Kambourakis, and C. Smiliotopoulos, "Let the cat out of the bag: Popular android iot apps under security scrutiny," *Sensors*, vol. 22, no. 2, p. 513, 2022.
- [27] H. Liu, T. Spink, and P. Patras, "Uncovering security vulnerabilities in the belkin wemo home automation ecosystem," in *2019 IEEE international conference on pervasive computing and communications workshops (PerCom workshops)*. IEEE, 2019, pp. 894–899.
- [28] S. Neupane, F. Baez, U. Paudel, F. V. Baez, M. Adamjee, L. De Carli, S. Das, and I. Ray, "On the data privacy, security, and risk postures of iot mobile companion apps," in *IFIP Annual Conference on Data and Applications Security and Privacy*. Springer, 2022, pp. 162–182.
- [29] K. Alghamdi, A. Alqazzaz, A. Liu, and H. Ming, "Iotverif: An automated tool to verify ssl/tls certificate validation in android mqtt client applications," in *Proceedings of the Eighth ACM Conference on data and application security and privacy*, 2018, pp. 95–102.
- [30] R. Alharbi and D. Aspinall, "An iot analysis framework: An investigation of iot smart cameras' vulnerabilities," in *Living in the Internet of Things: Cybersecurity of the IoT-2018*. IET, 2018, pp. 1–10.
- [31] M. Casagrande, E. Losiouk, M. Conti, M. Payer, and D. Antonioli, "Breakmi: Reversing, exploiting and fixing xiaomi fitness tracking ecosystem," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 330–366, 2022.
- [32] A. Allen, A. Mylonas, S. Vidalis, and D. Gritzalis, "Security evaluation of companion android applications in iot: The case of smart security devices," *Sensors*, vol. 24, no. 17, p. 5465, 2024.
- [33] T. Gaber, A. El-Ghamry, and A. E. Hassanien, "Injection attack detection using machine learning for smart iot applications," *Physical Communication*, vol. 52, p. 101685, 2022.
- [34] Y. Xu, D. Li, Q. Li, and S. Xu, "Malware evasion attacks against iot and other devices: An empirical study," *Tsinghua Science and Technology*, vol. 29, no. 1, pp. 127–142, 2023.
- [35] D. M. Junior, L. Melo, H. Lu, M. d' Amorim, and A. Prakash, "A study of vulnerability analysis of popular smart devices through their companion apps," in *2019 IEEE Security and Privacy Workshops (SPW)*. IEEE, 2019, pp. 181–186.

³Ptidej repository <https://www.ptidej.net/downloads/replications/iotj25c/>

- [36] S. Shasha, M. Mahmoud, M. Mannan, and A. Youssef, "Playing with danger: A taxonomy and evaluation of threats to smart toys," *IEEE Internet of Things Journal*, vol. 6, no. 2, pp. 2986–3002, 2018.
- [37] J. Du, Z. Zhang, F. Xu, and W. Diao, "Living in the past: Analyzing ble iot devices based on mobile companion apps in old versions," in *2023 19th International Conference on Mobility, Sensing and Networking (MSN)*. IEEE, 2023, pp. 317–324.
- [38] A. Subahi and G. Theodorakopoulos, "Detecting iot user behavior and sensitive information in encrypted iot-app traffic," *Sensors*, vol. 19, no. 21, p. 4777, 2019.
- [39] J. Du, F. Xu, C. Zhang, Z. Zhang, X. Liu, P. Ren, W. Diao, S. Guo, and K. Zhang, "Identifying the ble misconfigurations of iot devices through companion mobile apps," in *2022 19th Annual IEEE International Conference on Sensing, Communication, and Networking (SECON)*. IEEE, 2022, pp. 343–351.
- [40] F. Wei, S. Roy, X. Ou, and Robby, "Amandroid: A precise and general inter-component data flow analysis framework for security vetting of android apps," *ACM Transactions on Privacy and Security (TOPS)*, vol. 21, no. 3, pp. 1–32, 2018.
- [41] X. Jin, S. Manandhar, K. Kafle, Z. Lin, and A. Nadkarni, "Understanding iot security from a market-scale perspective," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 1615–1629.
- [42] W. Zhang, Y. Meng, Y. Liu, X. Zhang, Y. Zhang, and H. Zhu, "Homomit: Monitoring smart home apps from encrypted traffic," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 1074–1088.
- [43] L. Verderame, D. Caputo, M. Migliardi, and A. Merlo, "Appiotte: an architecture for the security assessment of mobile-iot ecosystems," in *Web, Artificial Intelligence and Network Applications: Proceedings of the Workshops of the 34th International Conference on Advanced Information Networking and Applications (WAINA-2020)*. Springer, 2020, pp. 867–876.
- [44] K. Knorr and D. Aspinall, "Security testing for android mhealth apps," in *2015 IEEE Eighth International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, 2015, pp. 1–8.
- [45] A. Liu, A. Alqazzaz, H. Ming, and B. Dharmalingam, "Iotverif: Automatic verification of ssl/tls certificate for iot applications," *IEEE Access*, vol. 9, pp. 27 038–27 050, 2019.
- [46] J. Chen, W. Diao, Q. Zhao, C. Zuo, Z. Lin, X. Wang, W. C. Lau, M. Sun, R. Yang, and K. Zhang, "Iotfuzzer: Discovering memory corruptions in iot through app-based fuzzing," in *NDSS*, 2018, pp. 1–15.
- [47] W. Ding, S. Liao, L. Cheng, X. Mi, Z. Zhao, and H. Hu, "Command hijacking on voice-controlled iot in amazon alexa platform," in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, 2024, pp. 654–666.
- [48] J. Cui, L. Wang, X. Zhao, and H. Zhang, "Towards predictive analysis of android vulnerability using statistical codes and machine learning for iot applications," *Computer Communications*, vol. 155, pp. 125–131, 2020.
- [49] E. B. Karbab, M. Debbabi, A. Derhab, and D. Mouheb, "Maldozer: Automatic framework for android malware detection using deep learning," *Digital investigation*, vol. 24, pp. S48–S59, 2018.
- [50] H. Bagheri, A. Sadeghi, J. Garcia, and S. Malek, "Covert: Compositional analysis of android inter-app permission leakage," *IEEE transactions on Software Engineering*, vol. 41, no. 9, pp. 866–886, 2015.
- [51] C. Stevens, M. Alhanahnah, Q. Yan, and H. Bagheri, "Comparing formal models of iot app coordination analysis," in *Proceedings of the 3rd ACM SIGSOFT International Workshop on Software Security from Design to Deployment*, 2020, pp. 3–10.
- [52] X. Wang, Y. Sun, S. Nanda, and X. Wang, "Looking from the mirror: Evaluating iot device security through mobile companion apps," in *28th USENIX Security Symposium (USENIX Security 19)*. Santa Clara, CA: USENIX Association, Aug. 2019, pp. 1151–1167. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity19/presentation/wang-xueqiang>
- [53] OWASP Foundation, "OWASP Top 10:2021," <https://owasp.org/Top10/>, accessed: 2024-10-22.
- [54] Elsevier, "Engineering village — search and discovery platform to answer engineering questions," <https://www.elsevier.com/products/engineering-village>, accessed: 2021-02-25.

APPENDIX

CLASSIFICATION OF PRIMARY STUDIES SECURITY VULNERABILITIES BASED OWASP TOP 10 2021 TAXONOMY

VDA DETAILS PER PRIMARY STUDIES

CORRESPONDING VDA PER SECURITY VULNERABILITY DETAILED METRICS FOR VDAS' EFFECTIVENESS CRITERIA

TABLE VI
CRITICAL SECURITY VULNERABILITIES OF IoTMA MAPPED WITH OWASP TOP 10 CATEGORIES

N°	OWASP top 10 2021 Vulnerability issue	Specific issue	Total studies
1	A02:2021 Cryptographic Failures	• User credentials and location data exposure [40] • Weak or Unencrypted (hard-coded keys) communications [17], [27], [36], [35]; • Use of deprecated TLS versions [29], [27]; • Use of broken or risky cryptographic algorithms, clear text storage of sensitive information [32], [28], [26], [4]; • Improper certificate validation [28], [26]; • Unencrypted communication channels, unencrypted hotspots [36]; • MITM attacks triggering vulnerabilities [30], [44], [29] • Protocol weakness [24], [35]	17
2	A01:2021 Broken Access Control	• Unauthorized access and authentication flaws [17], [16], [21], [20], [19], [15], [18], [4], [23], [24], [22], [25]; • Over-privileged access [42], [47]; • Authentication bypass and backdoor accounts [52] • Pairing flaws [35]; • PII leakage due to access control failures [23], [24], [38], [30]; • Permission redelegation and leakage due to inadequate encryption mechanisms [19], [4], [50];	16
3	A07:2021 Identification and Authentication Failures	• Insecure passkey entry/ improper authentication mechanisms [22]; • No certificate validation despite SSL/TLS connections [27], [29]; • Insecure Bluetooth Authentication practices [17], [39]; • Insecure or improper TLS/SSL implementation [30], [29], [27], [31], [44], [40]; • Weak parental PII control [36].	9
4	A03:2021 Injection	• SQL injection-related vulnerabilities [32], [26], [33], [25], [34], [28]; • Path traversal [32]; • Memory corruption vulnerabilities (stack and heap-based buffer overflow) [46]; • Cross-site scripting [27].	8
5	A04:2021 Insecure Design	• Lack of firmware update capabilities [39]; • Incorrect default permissions [28], [26]; • Insecure firmware update protocols [24]; • Insecure attribute reads and writes [22].	6
6	A05:2021 Security Misconfiguration	• Insecure device and communication configurations [39], [22], [35]; • Use of static UUIDs enabling device fingerprinting [18], [39]; • Improper certificate validation [29]; • Incorrect default permissions [36].	6
7	A06:2021 Vulnerable and Outdated Components	• Outdated or deprecated libraries [26]; • Use of vulnerable or outdated components [39], [37]; • Vulnerable software and hardware components [52], [25]; • Legacy exploit mitigation techniques [26].	4
8	A09:2021 Security Logging and Monitoring	• Insertion of sensitive information into log files [28], [26]; • Information exposure [28]; • PII leakage due to bad logging mechanisms [30], [28]; • Bad logging mechanism [30]; • Tracking through unique IDs [36]; • Malware [49];	4

TABLE VII
CRITICAL SECURITY VULNERABILITIES OF IoTMAS MAPPED WITH OWASP TOP 10 CATEGORIES CONT.

N°	OWASP top 10 2021 Vulnerability issue	Specific issue	Total studies
9	Software and data integrity failures	• Null pointer deference [46]; • Missing or inadequate privacy policies, lack of encryption of data [44], [21]; • Repackaged apps bypassed integrity checks [25].	3
10	A10:2021 Server-Side Request Forgery	• URL redirect vulnerabilities [36]; • Serve Side Request Forgery [32].	2
11	Privacy-related vulnerabilities	• No local/remote PII protection [36]; • Unencrypted logged data [44], [26]; • Insufficient monitoring of communications and excessive data collection [28], [38]; • Information leakage due to privacy issues [23], [18]; • False detection injection [41]; • DoS and Distributed DoS attacks [17], [52]; • Hidden functionalities [4]; • Sensitive Key Functional Units (KFUs) [22]; • IoT safety properties violation (action conflicts and action loop) [51].	

TABLE VIII
CATEGORIZATION OF IOTMAS SECURITY VDAS

Study	Year	Primary detection approach	Specific Analysis method
[29]	2018	Hybrid (Static + Dynamic)	Combined APK analysis with runtime observation for SSL/TLS verification
[24]	2018	Hybrid (Static + Dynamic)	Reverse engineering of FitBit embedded code and dynamic embedded code analysis
[38]	2019	Machine Learning (ML)	Random Forest Classifier for packets containing sensitive information classification
[17]	2017	Hybrid	Static analysis of Dalvik byteCode and dynamic packet sniffing for smart wristbands
[40]	2018	Static analysis (binary/APK analysis)	performed Inter-components data flow analysis for Android security vetting
[36]	2018	Manual analysis (network based)	Using their threat classification model, authors performed case studies on eleven well-known smart toys, through two experimental setups where they used Wireshark to sniff network traffic initiated by the toys and their mobile apps
[48]	2020	Machine Learning	Statistical code metrics and supervised ML to predict Android security vulnerabilities
[20]	2018	Static analysis (APK files) and Formal methods	IoT source code translation into an Intermediate representation (IR), extraction of a state model from the IR and application of model checking to verify desired properties.
[27]	2019	Reverse engineering	Reverse engineer the Belkin WeMo mobile companion app by decompiling the app to recover its source code and configuration files
[46]	2018	Dynamic analysis, Fuzzing and runtime analysis	performed automatic fuzzing through companion app to discover memory corruption vulnerabilities.
[42]	2018	Hybrid: Static and Dynamic analysis, Network analysis	Extraction of Deterministic Finite Automata from SmartApps' source code to model their expected behavior and leverage wireless-side analysis to infer SmartApp activities.
[19]	2012	Static analysis (APK / binary)	built CHEX, a static app vetting tool for component hijacking vulnerabilities, which include permission leakage, unauthorized data access, spoofing intent.
[35]	2019	Hybrid (static and dynamic) analysis	Discovery of vulnerable path through static analysis by decompiling app binary (APK file) to inspect code for security flaws and communication protocol understanding through dynamic analysis.
[50]	2015	Formal analysis (statically analyzed the reverse engineered source code)	Extract the model through static analysis, by reverse engineering of app binaries (APK file) to extract formal specifications of each app's behavior, and formal Analysis which includes the use of lightweight formal analysis techniques to verify certain properties.
[45]	2019	Hybrid: static and dynamic analysis	Static analysis of APK files to identify and instrument the APIs relevant to IoT functionalities, and performs dynamic analysis by observing the behavior of the instrumented IoT application during runtime to identify vulnerabilities.
[15]	2021	Hybrid: static and dynamic analysis	performed APK files analysis to identify fuzzing functions that can potentially trigger vulnerabilities, and empirically analyzed and verified the behavior of identified functions through dynamic analysis.
[23]	2016	Dynamic analysis	empirical measurement through the collection of real Bluetooth Low Energy (BLE) traffic traces from fitness trackers, and sniffing BLE packets using protocol analyzers to identified privacy leaks and vulnerabilities.
[30]	2018	Hybrid: static and dynamic analysis	Analyzing the APK file to extract permissions and identify potential vulnerabilities; manually investigating abnormal inputs, web server security issues and inspect privacy policy.
[18]	2019	Static analysis (APK/binary)	designed a tool, BleScope, that scans APK files to extract static UUIDs, detect BLE IoT devices that are vulnerable to fingerprinting, eavesdropping, and unauthorized access, such as insecure pairing methods, and identify weak or missing app-level authentication.
[43]	2020	Hybrid: static and dynamic analysis	used static and dynamic analysis for mobile IoT ecosystem vulnerabilities detection.
[44]	2015	Hybrid: Static and dynamic analysis	Examining APK files to identify potential vulnerabilities such as improper SSL usage, investigate whether abnormal and illegal inputs are accepted by the app and analyze their exportation and storage.
[16]	2019	Static (APK) analysis	Reverse-engineered firmware to understand the internal logic of devices and extract sensitive information, Man-in-the-middle monitoring to decrypt communications and black-box testing which mimic real devices to interact with the smart home platform.
[41]	2022	Static (APK analysis) analysis	Leveraged automated tools and large-scale code inspection for mobile IoT apps security
[4]	2019	Reverse engineering and manual analysis	reverse engineering of application binaries to identify insecure data handling,..., and dynamic analysis through sand-boxing and analyzed network data
[21]	2016	Static (source code) analysis	analyzed source code to identify privilege separation issues and weaknesses in protecting sensitive data.
[25]	2019	Dynamic analysis	Performed attacks, using cyber Kill Chain methodology to analyze the attack model, which includes phases like reconnaissance, weaponization, delivery, exploitation, installation, command and control, and actions on objectives, to reveal the ease with which Android apps can be tampered via repackaging, allowing attackers to inject malicious code.
[51]	2020	Formal method (formal verification through model checking, implemented via Alloy)	Compared state-based and rule-based modeling approaches using Alloy. They evaluated the scalability and effectiveness of both approaches in formally modeling and analyzing app coordination in IoT platforms, seeking to identify potential safety and security violations.

TABLE IX
CATEGORIZATION OF IOTMAS SECURITY VDAS CONT.

Study	Year	Primary detection approach	Specific Analysis Tool
[49]	2018	Machine Learning	Applied Deep learning to detect Android malware using a Deep learning sequence classifier.
[26]	2022	Hybrid: static and dynamic analysis	Full static and half dynamical analysis of 40 android apps belonging to 6 mainstream categories, paired to real-life IoT devices; in search of security vulnerabilities and flaws.
[37]	2023	Static (firmware binaries) analysis	Firmware updating capabilities analysis of BLE-enabled IoT devices based on their outdated companion App with a framework authors implemented, BLESecurAssist, which takes an IoT app as input and retrieves its outdated version, and a manual analysis of code protection techniques.
[34]	2023	Formal analysis and machine learning	Systematic analysis of evasion attacks-obfuscation and adversarial attack techniques-considering two Android malware datasets -Drebin and Androzoo- targeting edge computing devices' malware into functionality-preserving variants. Then conduct of a large-scale empirical study.
[28]	2022	Static (APK) analysis	Perform a latitudinal study and analysis of a sample of 455 IoTMAS to understand their privacy posture using various methods, focusing on three aspects: data privacy, security, and risk. Mostly used Mobile Security Framework (MobSF) to identify these vulnerabilities.
[22]	2023	Hybrid: static and dynamic analysis	designed BleGuide, a framework which employs a mobile app-centric approach to identify security vulnerabilities in BLE peripherals through the UUIDs used with BLE data, to underpin data transmissions in BLE, to glean rich information regarding device functionality and the underlying security issues. This information is then combined with information from app descriptions and BLE libraries, to identify the corresponding security vulnerabilities in BLE devices and determine the security or privacy impact they could have depending on the device functionality.
[33]	2022	Machine learning	Perform two types of feature selection techniques (constant removal and recursive feature limination), tested with a number of machine learning classifiers (i.e., SVM, Random Forest, and Decision Tree) to detect injection attacks.
[47]	2024	Dynamic analysis	design of a command hijacking system that targets Alexa's ranking process with three new attacks hijacking Alexa's built-in IoT commands, including Command Invocation Confounding Attack, Custom Command Attack, and Command-Intent Hijacking Attack.
[31]	2022	Reverse engineering and dynamic analysis	extensive analysis of all Xiaomi trackers since 2016 (i.e., Mi Band 2/3/4/5/6 and Amazfit Cor 2) and up-to-date Xiaomi mobile apps (Mi Fit and Zepp), using extensive static reverse-engineering and dynamic experiments. Then uncover severe specification-level vulnerabilities affecting the self-baked Xiaomi protocols; develop and evaluate six practical attacks on actual devices; developed breakmi, a security evaluation toolkit for fitness tracker ecosystems to automate our RE efforts and attacks.
[39]	2022	Static (APK) analysis	Static analysis of BLE mobile apps based on 5 predefined detection strategies summarised in five questions: 1.Does the device use a static Bluetooth MAC address, 2. Does the device use static UUIDs, 3. Does the device use an insecure pairing method, 4. Does the device use plaintext for transmission 5.Does the device use application layer authentication to investigate the wild status of security misconfiguration issues of the BLE deployments in the IoT ecosystem.
[52]	2019	Manual analysis	Companion app analysis using App Analysis Engine, which estimates the profile of an IoT device by analyzing its companion app and Cross-App Analysis Engine, which identifies correlations across different devices to build device families based on similarities in software, hardware, protocols, and backend services.
[32]	2024	Static (source code) analysis	Used static analysis tools (Ostorlab, MobSF, Snyk) to identify SQL injection, path traversal, and cryptographic vulnerabilities in smart security mobile apps and then manually inspected code for confirmation.

TABLE X
CONNECTION MATRIX BETWEEN VULNERABILITIES AND VDAS

Vulnerability	VDA	Studies	VDA validation metrics	Validation dataset
Unauthorized access and unauthorized data access	Hybrid (4), Static (Source code-1-, binary/APK-5-), dynamic analysis (2), RE (1)	[15], [16], [4], [17], [18], [19], [20], [21], [22], [23], [24], [25]	empirical validation with number of bugs found, comparison with baseline, real-world attack exploitability; number of discovered vulnerabilities, attack success rate, risk assessment, performance metrics (execution time, scalability, resource utilization), accuracy (TPR or recall / sensitivity, FPR, vulnerability Classification), reproducibility of the approach, quantitative metrics (property violation counts), detection success rate, qualitative attack demonstration, attack emulation Success, responsible disclosure to vendors and vulnerabilities acknowledgment	top Amazon smart homes applications, Google PlayStore apps, Android apps, SmartThings Github repos, Fitbit trackers and fitbit mobile apps, Android APKs from Google PlayStore...
Insecure or improper TLS/SSL implementation	Hybrid (3), Static (APK/binary) analysis (1), reverse engineering (2)	[29], [30], [27], [31], [44], [40]	empirical validation (test case-based evaluation, practical demonstration of the discovered vulnerabilities exploitation), API coverage, vulnerability detection success rate, attack success rate, reproducibility of the attack, Quantitative count of detected vulnerabilities, comparison with existing tools, responsible disclosure of vulnerabilities and vendor acknowledgment	real-world smart camera apps, Google Play and Amazon AppStore, Xiaomi trackers
SQL injection-related vulnerabilities	Hybrid (1), static APK analysis (1), dynamic analysis (1), ML (1), formal methods (1)	[32], [26], [33], [25], [34]	Attack success rate, real-world exploitability of detected vulnerabilities across apps, real-world attack implementation; precision, recall, accuracy, F1-score, TP, TN, FP, FN, comparative analysis with similar existing work; evasion rate, impact on code similarity detection, effectiveness comparison	ELM327 vehicle-related apps from Google Play store, AWID publicly available dataset of malware, 1,108 malware samples from Debrin and 1,762 malware sample from Androzoo
Personal Identifiable Information (PII) leakage due access control failures	Hybrid (2), dynamic (1) and ML (1)	[30], [24], [23], [38]	RF classification accuracy, real-world attack validation, attacks success rate, comparative analysis with existing work, real-world exploitability of detected vulnerabilities, comparison with baseline, decision tree classification accuracy	real-world devices from Amazon, network traffic traces from Fitbit devices and their mobile apps, Fitbit trackers apps
Use of broken or risky cryptographic algorithms and Cleartext storage	Hybrid (1), Static (Binary/APK-1- and source code -1-), RE (1)	[4], [26], [28], [32]	empirical validation (risk level assessment), attack success rate, vulnerability exploitability demonstration, Reproducibility of the approach, responsible disclosure and vendor acknowledgment, manual inspection by experts	Amazon apps, Smart toys, Huawei apps
Weak or Unencrypted communication	Hybrid (2), Manual (1), RE (1)	[35], [17], [36], [27]	Manual analysis, exploit-based execution, real-world attack validation, attack success rate, vendor confirmation, attack demonstration, attack emulation success, reproducibility of the approach, responsive disclosure and vendor acknowledgment	Amazon real-world devices, BLE packet traces from smart wristbands and Huawei watch, popular smart toys apps
Information leakage due to insecure design of BLE	Hybrid (1), dynamic (1), Machine learning (1), Static APK/binary (1)	[23], [24], [38], [37]	decision tree classification accuracy; RF classification accuracy, proof-of-concept attacks, reproducibility of the approach, manual verification, responsible disclosure and Fitbit vendor acknowledgment, large-scale measurements	Androzoo apps, encrypted network traffic traces, fitbit trackers and apps

TABLE XI
CONNECTION MATRIX BETWEEN VULNERABILITIES AND VDAS CONT.

Vulnerability	VDA	Studies	VDA validation metrics	Validation dataset
Insecure device and communication configurations	Hybrid (2), Static (Binary/APK) (1), dynamic (1)	[35], [22], [39], [25]	vulnerabilities prevalence rate, manual verification, case studies and impact assessment firmware analysis, live device attack, false positive analysis, property violations count, vulnerabilities exploitability demonstration	Google Play Store's BLE-enabled Android APK apps, 18 Android app marketplaces apps (excluding Google Play Store)
Permission redelegation and leakage due to inadequate encryption	Static (Binary/APK) (1), Formal (1), RE (1)	[4], [18], [50]	Empirical validation (risk level assessment, number of warning generated), quantitative metrics, attack success rate and real-world impact validation, manual validation and categorization, true positive rate, and counts of found vulnerabilities, real-world impact	apps from Google Play Store, F-Droid open source repository, Bazaar and MalGenome
MITM attacks triggering vulnerabilities	Hybrid (3)	[29], [30], [44]	test case-based validation, attacks success rate, reproducibility of the approach, API coverage and vulnerability detection success rate	top Google Play and Amazon Apps Store apps, mHealth apps for diabetes and blood pressure, network traffic traces from smart camera and their mobile apps

TABLE XII
VDAS' EFFECTIVENESS CRITERIA

N° VDAs	Vulnerability types	Validation metrics	Validation datasets
1 Hybrid analysis [42], [35], [29], [15], [30], [43], [44], [26], [22], [45], [17], [24]	- Information leakage due to insecure BLE design - Injection and SQL-injection-related vulnerabilities - Insecure device and communication configurations - Insecure or improper TLS/SSL implementation - MITM attacks triggering vulnerabilities - PII leakage due to access control failure - Unauthorized access and unauthorized data access - Use of broken or risky cryptographic algorithms and cleartext storage - Weak or unencrypted communication.	- Empirical validation (e.g., vulnerabilities detection rate, vulnerabilities-related attacks success rate, real-world exploitability for the vulnerability) - Quantitative evaluation - Qualitative validation - Responsible disclosure of vulnerabilities to vendors and vendors acknowledgment	Top Amazon smart homes applications, Google PlayStore apps, Android apps, SmartThings Github repos, Fitbit trackers and fitbit mobile apps, Android APKs from Google PlayStore
2 Static analysis with AP-K/binary files [18], [19], [40], [28], [41], [37], [39], [20], [16], [4], [27], [31], [32]	- Information leakage due to insecure BLE design - Insecure device and communication configurations - Insecure or improper TLS/SSL implementation - Permission redelegation and leakage due to inadequate encryption - Unauthorized access and unauthorized data access - Use of broken or risky cryptographic algorithms and cleartext storage - Weak or unencrypted communication.	- Empirical validation (e.g., vulnerabilities detection rate, vulnerabilities-related attacks success rate, number of vulnerable apps, real-world exploitability for the vulnerability, risk level assessment) - Quantitative evaluation - Technical performance, and accuracy validation - Comparison with existing tools - Analysis execution time per app - Manual case studies demonstration	18,166 free BLE IoT apps, 5,486 popular Android apps from official markets, 499 smartApps from SmartThings Apps market, smart cameras Apps, 18,166 apps from Google Play Store, 65 apps from smartThings market with 17 custom apps with 20 violations, 5 widely-used smart home platforms, simulated data from a google personal and business account, 154 apps from mHealth market for diabetes
3 Dynamic analysis [23], [25], [46], [47]	- Information leakage due to insecure BLE design - Insecure device and communication configurations - Permission redelegation and leakage due to inadequate encryption - Unauthorized access and unauthorized data access.	Empirical validation (e.g., percentage of flawed IoT devices, user activity detection accuracy, real-world attacks demonstration)	Fitbit data traces collected from a gymnasium, 213 Android apps from Play Store with 10 most popular vehicle control diagnostic apps.
4 Machine Learning [48], [49], [33], [38]	- Information leakage due to insecure BLE design - Insecure or improper TLS/SSL - PII leakage due to access control failure.	- Technical performance, and accuracy validation (e.g., accuracy rate of correctly identified vulnerabilities, precision / recall, F1-score) - Comparative analysis	AWID, network traffic collected from IoT companion apps

TABLE XIII
VDAS' EFFECTIVENESS CRITERIA CON.T

N° VDAs	Vulnerability types	Validation metrics	Validation datasets
5 Formal methods analysis [50], [34], [51]	• Insecure or improper TLS/SSL implementation • Permission redelegation and leakage due to inadequate encryption.	• Empirical validation (e.g.,count of generated warnings, manual review of generated warning for vulnerability confirmation, vulnerability count, malware detection success rate) • Technical performance (e.g.,TPR) • Comparison with existing tools	50 Android apps from F-Droid and Mal-Genome, Debrin with 1,108 malware samples, and 1,762 from Androzoo.
6 Static analysis with source code [21]	• Unauthorized access and unauthorized data access • Use of broken or risky cryptographic algorithms and cleartext storage.	• Qualitative validation (e.g.,proof-of-concept attacks demonstration, developer behavior)	499 smartapps and 132 IoT device trackers
7 Manual analysis [36], [52]	• Weak or unencrypted communication.	• Empirical validation (e.g.,experimentation attacks reproducibility, responsible disclosure of vulnerabilities to vendor) • Qualitative validation (e.g.,manual verification of vulnerabilities)	